Foodie - Complete Source Code

Repository: github.com/Manoj123312/glvc-capstoneproject (branch capstone-project)

Table of contents

README.md
frontend/README.md
backend/Dockerfile
backend/pom.xml
backend/src/main/java/com/foodie/FoodieApplication.java
backend/src/main/java/com/foodie/config/DataSeeder.java
backend/src/main/java/com/foodie/config/OpenApiConfig.java
backend/src/main/java/com/foodie/config/SecurityConfig.java
backend/src/main/java/com/foodie/controller/AuthController.java
backend/src/main/java/com/foodie/controller/DishController.java
backend/src/main/java/com/foodie/controller/OrderController.java
backend/src/main/java/com/foodie/controller/RestaurantController.java
backend/src/main/java/com/foodie/controller/UserController.java
backend/src/main/java/com/foodie/dto/Dtos.java
backend/src/main/java/com/foodie/entity/Dish.java
backend/src/main/java/com/foodie/entity/OrderEntity.java
backend/src/main/java/com/foodie/entity/OrderItem.java
backend/src/main/java/com/foodie/entity/Restaurant.java
backend/src/main/java/com/foodie/entity/User.java
backend/src/main/java/com/foodie/exception/BadRequestException.java
backend/src/main/java/com/foodie/exception/GlobalExceptionHandler.java
backend/src/main/java/com/foodie/exception/NotFoundException.java
backend/src/main/java/com/foodie/repository/DishRepository.java
backend/src/main/java/com/foodie/repository/OrderRepository.java
backend/src/main/java/com/foodie/repository/RestaurantRepository.java
backend/src/main/java/com/foodie/repository/UserRepository.java
backend/src/main/java/com/foodie/security/AppUserDetailsService.java
backend/src/main/java/com/foodie/security/JwtAuthFilter.java
backend/src/main/java/com/foodie/security/JwtService.java
backend/src/main/java/com/foodie/service/AuthService.java
backend/src/main/java/com/foodie/service/DishService.java
backend/src/main/java/com/foodie/service/OrderService.java
backend/src/main/java/com/foodie/service/RestaurantService.java
backend/src/main/java/com/foodie/service/UserService.java
backend/src/main/resources/application-h2.properties
backend/src/main/resources/application-mysql.properties
backend/src/main/resources/application.properties
backend/src/test/java/com/foodie/service/DishServiceTest.java
backend/src/test/java/com/foodie/service/OrderServiceTest.java
backend/src/test/java/com/foodie/service/UserAuthServiceTest.java
database/schema.sql
frontend/.gitignore
frontend/Dockerfile
frontend/angular.json
frontend/nginx.conf
frontend/package.json
frontend/src/app/app.component.ts
frontend/src/app/app.config.ts
frontend/src/app/app.routes.ts
frontend/src/app/core/config.ts
frontend/src/app/core/guards/auth.guard.ts

```
frontend/src/app/core/guards/auth.interceptor.ts
frontend/src/app/core/models.ts
frontend/src/app/core/services/api.services.ts
frontend/src/app/core/services/auth.service.ts
frontend/src/app/core/services/cart.service.ts
frontend/src/app/pages/admin/admin-dishes.component.ts
frontend/src/app/pages/admin/admin-orders.component.ts
frontend/src/app/pages/admin/admin-restaurants.component.ts
frontend/src/app/pages/admin/admin-users.component.ts
frontend/src/app/pages/auth/login.component.ts
frontend/src/app/pages/auth/register.component.ts
frontend/src/app/pages/user/cart.component.ts
frontend/src/app/pages/user/home.component.ts
frontend/src/app/pages/user/orders.component.ts
frontend/src/app/pages/user/profile.component.ts
frontend/src/main.ts
frontend/src/styles.css
.gitignore
Jenkinsfile
docker-compose.prod.yml
docker-compose.yml
```

README.md

```
# ■ Foodie – Food Delivery Full■Stack Capstone (Angular + Spring Boot + MySQL)

| Layer | Tech |
|---|---|
| Frontend | Angular 18 (standalone components, signals, HttpClient, route guards, JWT interceptor) |
| Backend | Java 17, Spring Boot 3.3, Spring Security (JWT), Spring Data JPA, Swagger/OpenAPI, JUnit 5 + Mockito |
| Database | MySQL 8 (or in■memory H2 for zero■setup local run) |
| DevOps | Docker, Docker Compose, Jenkins pipeline, AWS EC2 |

## Features
**Admin** (login `admin@foodie.com` / `admin123`): user management (change role, enable/disable, delete), restaurant management, product (dish) management, view all orders and update order status.
**User** (login `user@foodie.com` / `user123`, or register): browse restaurants & dishes, filter (restaurant, category, veg/non■veg, max price, search), add to cart, place order, order history with status, cancel (while Placed/Confirmed), edit profile.

## Backend structure
`entity` → `repository` → `service` (JUnit tests in `src/test`) → `controller` (Swagger annotated) · `security` (JWT) · `config` (CORS open, OpenAPI, data seeder) · `exception` (global handler).

Main endpoints (full list in Swagger):
| Method | URL | Access |
|---|---|---|
| POST | `/api/auth/register`, `/api/auth/login` | public |
| GET | `/api/restaurants`, `/api/dishes?restaurantId=&category=&veg=&maxPrice=&q=` | public |
| POST/PUT/DELETE | `/api/restaurants`, `/api/dishes` | ADMIN |
| GET/PUT/DELETE | `/api/admin/users` | ADMIN |
| GET / PUT | `/api/admin/orders`, `/api/admin/orders/{id}/status` | ADMIN |
| POST / GET | `/api/orders`, `/api/orders/my` ; PUT `/api/orders/{id}/cancel` | USER |
| GET / PUT | `/api/users/me` | logged in |

Swagger UI: http://localhost:8080/swagger-ui.html (click **Authorize**, paste the token from `/api/auth/login`).

---
## 1. Download the project
```bash
git clone -b capstone-project https://github.com/Manoj123312/glvc-capstoneproject.git
cd glvc-capstoneproject
```
(or on GitHub switch to the `capstone-project` branch → **Code → Download ZIP**, then extract.)

2. Prerequisites (install once)
Tool	Version	Check
JDK	17 or newer	`java -version`
Maven	3.9+ (or use the IDE's bundled Maven)	`mvn -v`
Node.js	18.19+ / 20 LTS	`node -v`
Angular CLI	18	`npm i -g @angular/cli@18` then `ng version`
MySQL 8	*optional* – only for MySQL mode	`mysql --version`
Docker Desktop	*optional* – for container run	`docker -v`

3. Open in an IDE
Backend – IntelliJ IDEA: `File → Open` → select the `backend` folder (the one containing `pom.xml`) → *Trust Project* → wait for Maven import. Set `File → Project Structure → SDK` to JDK 17+. Open `FoodieApplication.java` and click ■ **Run**.
Backend – Eclipse/STS: `File → Import → Maven → Existing Maven Projects` → choose `backend` → Finish → right■click `FoodieApplication` → *Run As → Java Application*.
Frontend – VS Code: `File → Open Folder…` → choose `frontend`. Open the terminal (`Ctrl+\`).

4. Run locally (no database needed – H2 default)
Terminal 1 – backend
```bash
cd backend
mvn spring-boot:run
```
Wait for `Started FoodieApplication`. API: http://localhost:8080 · Swagger: http://localhost:8080/swagger-ui.html
(Sample admin, user, 3 restaurants and 8 dishes are seeded automatically. H2 data resets on every restart.)

Terminal 2 – frontend
```bash
cd frontend
npm install
ng serve
```
Open http://localhost:4200 .

Run with MySQL instead of H2
1. Start MySQL and make sure root password matches (default `root`), or set env vars `DB_USERNAME`, `DB_PASSWORD`, `DB_URL`.
2. (Optional) run `database/schema.sql`; otherwise the app creates the DB and tables itself.
3. Start backend with the `mysql` profile:
```bash
cd backend
mvn spring-boot:run -Dspring-boot.run.profiles=mysql
```
(IntelliJ: Run configuration → *Active profiles* = `mysql`.)

Run unit tests
```bash
```
```

```
cd backend && mvn test
```

5. Run everything with Docker (MySQL + backend + frontend)
```bash
docker compose up --build
```
Frontend → http://localhost · Backend → http://localhost:8080 · Swagger → http://localhost:8080/swagger-ui.html

6. CI/CD – Jenkins → Docker Hub → AWS EC2
1. EC2 (Amazon Linux/Ubuntu): install Docker + Compose plugin; open inbound ports **22, 80, 8080** in the security group.
2. Jenkins: install plugins *Pipeline, Git, Docker Pipeline, SSH Agent*; add credentials `dockerhub-creds` (Docker Hub user/password) and `ec2-ssh-key` (EC2 private key). Jenkins host needs JDK 17, Maven, Node 20 and Docker.
3. Edit `DOCKER_USER` and `EC2_HOST` at the top of `Jenkinsfile`, create a *Pipeline from SCM* job pointing to this repo.
4. Build: tests → Angular build → Docker images → push → `docker compose up -d` on EC2 using `docker-compose.prod.yml`. App is on at `http://<EC2-public-ip>`.
5. Production: set a strong `JWT_SECRET` and `MYSQL_ROOT_PASSWORD` on the EC2 host and change the seeded admin password.

7. Troubleshooting
Problem	Fix
`Port 8080 already in use`	Stop the other app, or run with `--server.port=8081` (and update `frontend/src/app/core/config.ts`.
`release version 17 not supported`	Your JDK is older than 17 – install JDK 17+.
`Access denied for user 'root'` (MySQL)	Set `DB_PASSWORD` to your MySQL root password.
Frontend shows "Cannot reach the server"	Backend isn't running on :8080.
`npm install` errors	Use Node 18.19+ / 20 LTS; delete `node_modules` and retry.
`ng: command not found`	`npm i -g @angular/cli@18`, or use `npx ng serve`.

The frontend calls the backend at `http://<current-host>:8080/api` (see `frontend/src/app/core/config.ts`); CORS is open on the backend.

frontend/README.md

```
# Frontend

This project was generated with [Angular CLI](https://github.com/angular/angular-cli) version 18.2.21.

## Development server

Run `ng serve` for a dev server. Navigate to `http://localhost:4200/`. The application will automatically reload if you change any of the source files.

## Code scaffolding

Run `ng generate component component-name` to generate a new component. You can also use `ng generate directive|pipe|service|class|guard|interface|enum|module`.

## Build

Run `ng build` to build the project. The build artifacts will be stored in the `dist/` directory.

## Running unit tests

Run `ng test` to execute the unit tests via [Karma](https://karma-runner.github.io).

## Running end-to-end tests

Run `ng e2e` to execute the end-to-end tests via a platform of your choice. To use this command, you need to first add a package that implements end-to-end testing capabilities.

## Further help

To get more help on the Angular CLI use `ng help` or go check out the [Angular CLI Overview and Command Reference](https://angular.dev/tools/cli) page.
```

backend/Dockerfile

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q -B dependency:go-offline
COPY src ./src
RUN mvn -q -B -DskipTests package

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/foodie-backend-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

backend/pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
    <relativePath/>
  </parent>
  <groupId>com.foodie</groupId>
  <artifactId>foodie-backend</artifactId>
  <version>1.0.0</version>
  <name>foodie-backend</name>

  <properties>
    <java.version>17</java.version>
    <jjwt.version>0.12.6</jjwt.version>
  </properties>

  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-security</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-validation</artifactId></dependency>
    <dependency><groupId>org.springdoc</groupId><artifactId>springdoc-openapi-starter-webmvc-ui</artifactId><version>2.6.0</vers
n></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>${jjwt.version}</version></dependenc
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId><version>${jjwt.version}</version><scope>run
me</scope></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId><version>${jjwt.version}</version><scope>
ntime</scope></dependency>
    <dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-j</artifactId><scope>runtime</scope></dependency>
    <dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><scope>runtime</scope></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId><scope>test</scope><
ependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

backend/src/main/java/com/foodie/FoodieApplication.java

```java
package com.foodie;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class FoodieApplication {
    public static void main(String[] args) {
        SpringApplication.run(FoodieApplication.class, args);
    }
}
```

backend/src/main/java/com/foodie/config/DataSeeder.java

```java
package com.foodie.config;

import com.foodie.entity.Dish;
import com.foodie.entity.Restaurant;
import com.foodie.entity.User;
import com.foodie.repository.DishRepository;
import com.foodie.repository.RestaurantRepository;
import com.foodie.repository.UserRepository;
import java.math.BigDecimal;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class DataSeeder {
```

```

    @Bean
    CommandLineRunner seed(UserRepository users, RestaurantRepository restaurants, DishRepository dishes, PasswordEncoder enc) {
        return args -> {
            if (!users.existsByEmail("admin@foodie.com")) {
                users.save(user("Admin", "admin@foodie.com", "admin123", User.Role.ADMIN, enc));
                users.save(user("Demo User", "user@foodie.com", "user123", User.Role.USER, enc));
            }
            if (restaurants.count() == 0) {
                Restaurant r1 = restaurant(restaurants, "Spice Garden", "Indian", "Hyderabad", "Banjara Hills", 4.5,
                        "https://images.unsplash.com/photo-1517248135467-4c7edcad34c4?w=600");
                Restaurant r2 = restaurant(restaurants, "Pizza Palace", "Italian", "Hyderabad", "Madhapur", 4.2,
                        "https://images.unsplash.com/photo-1513104890138-7c749659a591?w=600");
                Restaurant r3 = restaurant(restaurants, "Dragon Wok", "Chinese", "Hyderabad", "Gachibowli", 4.0,
                        "https://images.unsplash.com/photo-1525755662778-989d0524087e?w=600");
                dish(dishes, r1, "Chicken Biryani", "Hyderabadi dum biryani", "249", "Main Course", false, "https://images.unspl
h.com/photo-1563379091339-03b21ab4a4f8?w=600");
                dish(dishes, r1, "Paneer Butter Masala", "Creamy tomato gravy", "199", "Main Course", true, "https://images.unsp
sh.com/photo-1631452180519-c014fe946bc7?w=600");
                dish(dishes, r1, "Gulab Jamun", "Sweet milk dumplings", "89", "Dessert", true, "https://images.unsplash.com/phot
1666190077424-8c6d5c3d3f7e?w=600");
                dish(dishes, r2, "Margherita Pizza", "Classic cheese pizza", "299", "Pizza", true, "https://images.unsplash.com/
oto-1574071318508-1cdbab80d002?w=600");
                dish(dishes, r2, "Pepperoni Pizza", "Spicy pepperoni", "379", "Pizza", false, "https://images.unsplash.com/photo
628840042765-356cda07504e?w=600");
                dish(dishes, r2, "Garlic Bread", "Toasted with butter", "119", "Starter", true, "https://images.unsplash.com/pho
-1573140401552-3fab0b24306f?w=600");
                dish(dishes, r3, "Veg Hakka Noodles", "Wok tossed noodles", "169", "Noodles", true, "https://images.unsplash.com
hoto-1585032226651-759b368d7246?w=600");
                dish(dishes, r3, "Chicken Manchurian", "Indo-Chinese gravy", "229", "Main Course", false, "https://images.unspla
.com/photo-1603133872878-684f208fb84b?w=600");
            }
        };
    }

    private User user(String name, String email, String pw, User.Role role, PasswordEncoder enc) {
        User u = new User();
        u.setName(name);
        u.setEmail(email);
        u.setPassword(enc.encode(pw));
        u.setRole(role);
        return u;
    }

    private Restaurant restaurant(RestaurantRepository repo, String name, String cuisine, String city, String address,
                                  double rating, String img) {
        Restaurant r = new Restaurant();
        r.setName(name);
        r.setCuisine(cuisine);
        r.setCity(city);
        r.setAddress(address);
        r.setRating(rating);
        r.setImageUrl(img);
        return repo.save(r);
    }

    private void dish(DishRepository repo, Restaurant r, String name, String desc, String price, String cat,
                      boolean veg, String img) {
        Dish d = new Dish();
        d.setRestaurant(r);
        d.setName(name);
        d.setDescription(desc);
        d.setPrice(new BigDecimal(price));
        d.setCategory(cat);
        d.setVeg(veg);
        d.setImageUrl(img);
        repo.save(d);
    }
}
```

backend/src/main/java/com/foodie/config/OpenApiConfig.java

```
package com.foodie.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {
    @Bean
    OpenAPI foodieOpenApi() {
```

```
        return new OpenAPI()
                .info(new Info().title("Foodie Food Delivery API").version("1.0")
                        .description("Click 'Authorize' and paste the JWT returned by /api/auth/login"))
                .addSecurityItem(new SecurityRequirement().addList("bearerAuth"))
                .components(new Components().addSecuritySchemes("bearerAuth",
                        new SecurityScheme().type(SecurityScheme.Type.HTTP).scheme("bearer").bearerFormat("JWT")));
    }
}
```

backend/src/main/java/com/foodie/config/SecurityConfig.java

```
package com.foodie.config;

import com.foodie.security.JwtAuthFilter;
import java.util.List;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.http.HttpStatus;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http, JwtAuthFilter jwtFilter) throws Exception {
        http.csrf(c -> c.disable())
            .cors(c -> c.configurationSource(corsConfigurationSource()))
            .headers(h -> h.frameOptions(f -> f.sameOrigin()))
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(e -> e.authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED))
                .accessDeniedHandler((req, res, ex) -> res.setStatus(HttpStatus.FORBIDDEN.value())))
            .authorizeHttpRequests(a -> a
                .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
                .requestMatchers("/api/auth/**", "/swagger-ui/**", "/swagger-ui.html", "/v3/api-docs/**", "/h2-console/**", "/ac
ator/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/restaurants/**", "/api/dishes/**").permitAll()
                .requestMatchers("/api/restaurants/**", "/api/dishes/**", "/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    // CORS is fully open as required by the course.
    @Bean
    CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration cfg = new CorsConfiguration();
        cfg.setAllowedOriginPatterns(List.of("*"));
        cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
        cfg.setAllowedHeaders(List.of("*"));
        UrlBasedCorsConfigurationSource src = new UrlBasedCorsConfigurationSource();
        src.registerCorsConfiguration("/**", cfg);
        return src;
    }

    @Bean
    PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    AuthenticationManager authenticationManager(AuthenticationConfiguration c) throws Exception {
        return c.getAuthenticationManager();
    }
}
```

backend/src/main/java/com/foodie/controller/AuthController.java

```
package com.foodie.controller;

import com.foodie.dto.Dtos.*;
import com.foodie.service.AuthService;
```

```
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
@Tag(name = "Authentication")
public class AuthController {
    private final AuthService auth;

    public AuthController(AuthService auth) { this.auth = auth; }

    @PostMapping("/register")
    @ResponseStatus(HttpStatus.CREATED)
    public AuthResponse register(@Valid @RequestBody RegisterRequest req) { return auth.register(req); }

    @PostMapping("/login")
    public AuthResponse login(@Valid @RequestBody LoginRequest req) { return auth.login(req); }
}
```

backend/src/main/java/com/foodie/controller/DishController.java

```
package com.foodie.controller;

import com.foodie.dto.Dtos.DishRequest;
import com.foodie.entity.Dish;
import com.foodie.service.DishService;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import java.math.BigDecimal;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/dishes")
@Tag(name = "Dishes")
public class DishController {
    private final DishService service;

    public DishController(DishService service) { this.service = service; }

    @GetMapping
    public List<Dish> search(@RequestParam(required = false) Long restaurantId,
                             @RequestParam(required = false) String category,
                             @RequestParam(required = false) Boolean veg,
                             @RequestParam(required = false) BigDecimal maxPrice,
                             @RequestParam(required = false) String q) {
        return service.search(restaurantId, category, veg, maxPrice, q);
    }

    @GetMapping("/{id}")
    public Dish one(@PathVariable Long id) { return service.findById(id); }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Dish create(@Valid @RequestBody DishRequest req) { return service.create(req); }

    @PutMapping("/{id}")
    public Dish update(@PathVariable Long id, @Valid @RequestBody DishRequest req) { return service.update(id, req); }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) { service.delete(id); }
}
```

backend/src/main/java/com/foodie/controller/OrderController.java

```
package com.foodie.controller;

import com.foodie.dto.Dtos.OrderDto;
import com.foodie.dto.Dtos.OrderRequest;
import com.foodie.dto.Dtos.StatusRequest;
import com.foodie.service.OrderService;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import java.security.Principal;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestController
```

```java
@Tag(name = "Orders")
public class OrderController {
    private final OrderService service;

    public OrderController(OrderService service) { this.service = service; }

    @PostMapping("/api/orders")
    @ResponseStatus(HttpStatus.CREATED)
    public OrderDto place(Principal p, @Valid @RequestBody OrderRequest req) { return service.placeOrder(p.getName(), req); }

    @GetMapping("/api/orders/my")
    public List<OrderDto> mine(Principal p) { return service.myOrders(p.getName()); }

    @PutMapping("/api/orders/{id}/cancel")
    public OrderDto cancel(Principal p, @PathVariable Long id) { return service.cancel(p.getName(), id); }

    @GetMapping("/api/admin/orders")
    public List<OrderDto> all() { return service.allOrders(); }

    @PutMapping("/api/admin/orders/{id}/status")
    public OrderDto updateStatus(@PathVariable Long id, @Valid @RequestBody StatusRequest req) {
        return service.updateStatus(id, req.status());
    }
}
```

backend/src/main/java/com/foodie/controller/RestaurantController.java

```java
package com.foodie.controller;

import com.foodie.entity.Restaurant;
import com.foodie.service.RestaurantService;
import io.swagger.v3.oas.annotations.tags.Tag;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/restaurants")
@Tag(name = "Restaurants")
public class RestaurantController {
    private final RestaurantService service;

    public RestaurantController(RestaurantService service) { this.service = service; }

    @GetMapping
    public List<Restaurant> all() { return service.findAll(); }

    @GetMapping("/{id}")
    public Restaurant one(@PathVariable Long id) { return service.findById(id); }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Restaurant create(@RequestBody Restaurant r) { return service.create(r); }

    @PutMapping("/{id}")
    public Restaurant update(@PathVariable Long id, @RequestBody Restaurant r) { return service.update(id, r); }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) { service.delete(id); }
}
```

backend/src/main/java/com/foodie/controller/UserController.java

```java
package com.foodie.controller;

import com.foodie.dto.Dtos.UserDto;
import com.foodie.dto.Dtos.UserUpdateRequest;
import com.foodie.service.UserService;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import java.util.List;
import java.security.Principal;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@Tag(name = "Users")
public class UserController {
    private final UserService users;

    public UserController(UserService users) { this.users = users; }

```

```
    // ---- Admin: user management ----
    @GetMapping("/api/admin/users")
    public List<UserDto> all() { return users.findAll(); }

    @GetMapping("/api/admin/users/{id}")
    public UserDto one(@PathVariable Long id) { return users.findById(id); }

    @PutMapping("/api/admin/users/{id}")
    public UserDto update(@PathVariable Long id, @Valid @RequestBody UserUpdateRequest req) { return users.update(id, req); }

    @DeleteMapping("/api/admin/users/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id, Principal principal) {
        users.delete(id, principal.getName());
        return ResponseEntity.noContent().build();
    }

    // ---- Logged-in user: own profile ----
    @GetMapping("/api/users/me")
    public UserDto me(Principal principal) { return users.findByEmail(principal.getName()); }

    @PutMapping("/api/users/me")
    public UserDto updateMe(Principal principal, @Valid @RequestBody UserUpdateRequest req) {
        return users.updateProfile(principal.getName(), req);
    }
}
```

backend/src/main/java/com/foodie/dto/Dtos.java

```
package com.foodie.dto;

import com.foodie.entity.OrderEntity;
import com.foodie.entity.User;
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.List;

public final class Dtos {
    private Dtos() {}

    public record RegisterRequest(@NotBlank String name, @NotBlank @Email String email,
                                  @NotBlank @Size(min = 6) String password, String phone, String address) {}

    public record LoginRequest(@NotBlank @Email String email, @NotBlank String password) {}

    public record AuthResponse(String token, Long id, String name, String email, String role) {}

    public record UserDto(Long id, String name, String email, String phone, String address, String role, boolean active) {
        public static UserDto from(User u) {
            return new UserDto(u.getId(), u.getName(), u.getEmail(), u.getPhone(), u.getAddress(),
                    u.getRole().name(), u.isActive());
        }
    }

    public record UserUpdateRequest(@NotBlank String name, String phone, String address, User.Role role, Boolean active) {}

    public record DishRequest(@NotNull Long restaurantId, @NotBlank String name, String description,
                              @NotNull @DecimalMin("0.0") BigDecimal price, String category, boolean veg,
                              String imageUrl, Boolean available) {}

    public record OrderItemRequest(@NotNull Long dishId, @Min(1) int quantity) {}

    public record OrderRequest(@NotBlank String deliveryAddress, @NotEmpty @Valid List<OrderItemRequest> items) {}

    public record StatusRequest(@NotNull OrderEntity.Status status) {}

    public record OrderItemDto(Long dishId, String dishName, String restaurantName, BigDecimal unitPrice, int quantity) {}

    public record OrderDto(Long id, Long userId, String userName, String status, BigDecimal totalAmount,
                           String deliveryAddress, LocalDateTime createdAt, List<OrderItemDto> items) {
        public static OrderDto from(OrderEntity o) {
            return new OrderDto(o.getId(), o.getUser().getId(), o.getUser().getName(), o.getStatus().name(),
                    o.getTotalAmount(), o.getDeliveryAddress(), o.getCreatedAt(),
                    o.getItems().stream().map(i -> new OrderItemDto(i.getDishId(), i.getDishName(),
                            i.getRestaurantName(), i.getUnitPrice(), i.getQuantity())).toList());
        }
    }
}
```

backend/src/main/java/com/foodie/entity/Dish.java

```
package com.foodie.entity;
```

```

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "dishes")
public class Dish {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String name;
    @Column(length = 500)
    private String description;
    @Column(nullable = false)
    private BigDecimal price;
    private String category;
    private boolean veg;
    private String imageUrl;
    private boolean available = true;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "restaurant_id")
    @JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
    private Restaurant restaurant;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }
    public boolean isVeg() { return veg; }
    public void setVeg(boolean veg) { this.veg = veg; }
    public String getImageUrl() { return imageUrl; }
    public void setImageUrl(String imageUrl) { this.imageUrl = imageUrl; }
    public boolean isAvailable() { return available; }
    public void setAvailable(boolean available) { this.available = available; }
    public Restaurant getRestaurant() { return restaurant; }
    public void setRestaurant(Restaurant restaurant) { this.restaurant = restaurant; }
}
```

backend/src/main/java/com/foodie/entity/OrderEntity.java

```

package com.foodie.entity;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "orders")
public class OrderEntity {
    public enum Status { PLACED, CONFIRMED, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELLED }

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "user_id")
    private User user;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.EAGER)
    private List<OrderItem> items = new ArrayList<>();

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Status status = Status.PLACED;

    @Column(nullable = false)
    private BigDecimal totalAmount = BigDecimal.ZERO;
    private String deliveryAddress;
    private LocalDateTime createdAt = LocalDateTime.now();

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }
    public List<OrderItem> getItems() { return items; }
    public void setItems(List<OrderItem> items) { this.items = items; }
```

```
    public Status getStatus() { return status; }
    public void setStatus(Status status) { this.status = status; }
    public BigDecimal getTotalAmount() { return totalAmount; }
    public void setTotalAmount(BigDecimal totalAmount) { this.totalAmount = totalAmount; }
    public String getDeliveryAddress() { return deliveryAddress; }
    public void setDeliveryAddress(String deliveryAddress) { this.deliveryAddress = deliveryAddress; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
}
```

backend/src/main/java/com/foodie/entity/OrderItem.java

```
package com.foodie.entity;

import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "order_items")
public class OrderItem {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false)
    @JoinColumn(name = "order_id")
    private OrderEntity order;

    private Long dishId;
    private String dishName;
    private String restaurantName;
    private BigDecimal unitPrice;
    private int quantity;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public OrderEntity getOrder() { return order; }
    public void setOrder(OrderEntity order) { this.order = order; }
    public Long getDishId() { return dishId; }
    public void setDishId(Long dishId) { this.dishId = dishId; }
    public String getDishName() { return dishName; }
    public void setDishName(String dishName) { this.dishName = dishName; }
    public String getRestaurantName() { return restaurantName; }
    public void setRestaurantName(String restaurantName) { this.restaurantName = restaurantName; }
    public BigDecimal getUnitPrice() { return unitPrice; }
    public void setUnitPrice(BigDecimal unitPrice) { this.unitPrice = unitPrice; }
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

backend/src/main/java/com/foodie/entity/Restaurant.java

```
package com.foodie.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "restaurants")
public class Restaurant {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String name;
    private String cuisine;
    private String city;
    private String address;
    private String imageUrl;
    private double rating;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getCuisine() { return cuisine; }
    public void setCuisine(String cuisine) { this.cuisine = cuisine; }
    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }
    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }
    public String getImageUrl() { return imageUrl; }
    public void setImageUrl(String imageUrl) { this.imageUrl = imageUrl; }
    public double getRating() { return rating; }
    public void setRating(double rating) { this.rating = rating; }
}
```

backend/src/main/java/com/foodie/entity/User.java

```
package com.foodie.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {
    public enum Role { ADMIN, USER }

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String name;
    @Column(nullable = false, unique = true)
    private String email;
    @Column(nullable = false)
    private String password;
    private String phone;
    private String address;
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role = Role.USER;
    private boolean active = true;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getPhone() { return phone; }
    public void setPhone(String phone) { this.phone = phone; }
    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }
    public Role getRole() { return role; }
    public void setRole(Role role) { this.role = role; }
    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }
}
```

backend/src/main/java/com/foodie/exception/BadRequestException.java

```
package com.foodie.exception;

public class BadRequestException extends RuntimeException {
    public BadRequestException(String message) { super(message); }
}
```

backend/src/main/java/com/foodie/exception/GlobalExceptionHandler.java

```
package com.foodie.exception;

import java.time.Instant;
import java.util.Map;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.security.authentication.DisabledException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    private ResponseEntity<Map<String, Object>> body(HttpStatus status, String message) {
        return ResponseEntity.status(status)
                .body(Map.of("timestamp", Instant.now().toString(), "status", status.value(), "message", message));
    }

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<Map<String, Object>> notFound(NotFoundException e) { return body(HttpStatus.NOT_FOUND, e.getMessage())
}
```

```
25      @ExceptionHandler(BadRequestException.class)
26      public ResponseEntity<Map<String, Object>> bad(BadRequestException e) { return body(HttpStatus.BAD_REQUEST, e.getMessage()); }
27
28      @ExceptionHandler({BadCredentialsException.class, DisabledException.class})
29      public ResponseEntity<Map<String, Object>> auth(RuntimeException e) { return body(HttpStatus.UNAUTHORIZED, "Invalid credentials or account disabled"); }
30
31      @ExceptionHandler(DataIntegrityViolationException.class)
32      public ResponseEntity<Map<String, Object>> integrity(DataIntegrityViolationException e) {
33          return body(HttpStatus.CONFLICT, "Operation violates data constraints (duplicate or referenced record)");
34      }
35
36      @ExceptionHandler(MethodArgumentNotValidException.class)
37      public ResponseEntity<Map<String, Object>> invalid(MethodArgumentNotValidException e) {
38          String msg = e.getBindingResult().getFieldErrors().stream()
39                  .map(f -> f.getField() + " " + f.getDefaultMessage()).reduce((a, b) -> a + "; " + b).orElse("Validation failed");
40          return body(HttpStatus.BAD_REQUEST, msg);
41      }
42  }
```

backend/src/main/java/com/foodie/exception/NotFoundException.java

```
1   package com.foodie.exception;
2
3   public class NotFoundException extends RuntimeException {
4       public NotFoundException(String message) { super(message); }
5   }
```

backend/src/main/java/com/foodie/repository/DishRepository.java

```
1   package com.foodie.repository;
2
3   import com.foodie.entity.Dish;
4   import java.math.BigDecimal;
5   import java.util.List;
6   import org.springframework.data.jpa.repository.JpaRepository;
7   import org.springframework.data.jpa.repository.Query;
8   import org.springframework.data.repository.query.Param;
9
10  public interface DishRepository extends JpaRepository<Dish, Long> {
11      @Query("""
12          select d from Dish d
13          where (:restaurantId is null or d.restaurant.id = :restaurantId)
14            and (:category is null or lower(d.category) = lower(:category))
15            and (:veg is null or d.veg = :veg)
16            and (:maxPrice is null or d.price <= :maxPrice)
17            and (:q is null or lower(d.name) like lower(concat('%', :q, '%')))
18          order by d.name
19          """)
20      List<Dish> search(@Param("restaurantId") Long restaurantId,
21                        @Param("category") String category,
22                        @Param("veg") Boolean veg,
23                        @Param("maxPrice") BigDecimal maxPrice,
24                        @Param("q") String q);
25
26      long countByRestaurantId(Long restaurantId);
27  }
```

backend/src/main/java/com/foodie/repository/OrderRepository.java

```
1   package com.foodie.repository;
2
3   import com.foodie.entity.OrderEntity;
4   import java.util.List;
5   import org.springframework.data.jpa.repository.JpaRepository;
6
7   public interface OrderRepository extends JpaRepository<OrderEntity, Long> {
8       List<OrderEntity> findByUserIdOrderByCreatedAtDesc(Long userId);
9       List<OrderEntity> findAllByOrderByCreatedAtDesc();
10  }
```

backend/src/main/java/com/foodie/repository/RestaurantRepository.java

```
1   package com.foodie.repository;
2
3   import com.foodie.entity.Restaurant;
4   import org.springframework.data.jpa.repository.JpaRepository;
```

```
5
6  public interface RestaurantRepository extends JpaRepository<Restaurant, Long> {
7  }
```

backend/src/main/java/com/foodie/repository/UserRepository.java

```
1  package com.foodie.repository;
2
3  import com.foodie.entity.User;
4  import java.util.Optional;
5  import org.springframework.data.jpa.repository.JpaRepository;
6
7  public interface UserRepository extends JpaRepository<User, Long> {
8      Optional<User> findByEmail(String email);
9      boolean existsByEmail(String email);
10 }
```

backend/src/main/java/com/foodie/security/AppUserDetailsService.java

```
1  package com.foodie.security;
2
3  import com.foodie.repository.UserRepository;
4  import org.springframework.security.core.userdetails.User;
5  import org.springframework.security.core.userdetails.UserDetails;
6  import org.springframework.security.core.userdetails.UserDetailsService;
7  import org.springframework.security.core.userdetails.UsernameNotFoundException;
8  import org.springframework.stereotype.Service;
9
10 @Service
11 public class AppUserDetailsService implements UserDetailsService {
12     private final UserRepository users;
13
14     public AppUserDetailsService(UserRepository users) { this.users = users; }
15
16     @Override
17     public UserDetails loadUserByUsername(String email) {
18         com.foodie.entity.User u = users.findByEmail(email)
19                 .orElseThrow(() -> new UsernameNotFoundException("User not found"));
20         return User.withUsername(u.getEmail()).password(u.getPassword()).roles(u.getRole().name())
21                 .disabled(!u.isActive()).build();
22     }
23 }
```

backend/src/main/java/com/foodie/security/JwtAuthFilter.java

```
1  package com.foodie.security;
2
3  import jakarta.servlet.FilterChain;
4  import jakarta.servlet.ServletException;
5  import jakarta.servlet.http.HttpServletRequest;
6  import jakarta.servlet.http.HttpServletResponse;
7  import java.io.IOException;
8  import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
9  import org.springframework.security.core.context.SecurityContextHolder;
10 import org.springframework.security.core.userdetails.UserDetails;
11 import org.springframework.stereotype.Component;
12 import org.springframework.web.filter.OncePerRequestFilter;
13
14 @Component
15 public class JwtAuthFilter extends OncePerRequestFilter {
16     private final JwtService jwt;
17     private final AppUserDetailsService details;
18
19     public JwtAuthFilter(JwtService jwt, AppUserDetailsService details) {
20         this.jwt = jwt;
21         this.details = details;
22     }
23
24     @Override
25     protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
26             throws ServletException, IOException {
27         String header = req.getHeader("Authorization");
28         if (header != null && header.startsWith("Bearer ") && SecurityContextHolder.getContext().getAuthentication() == null) {
29             try {
30                 UserDetails ud = details.loadUserByUsername(jwt.extractEmail(header.substring(7)));
31                 if (ud.isEnabled()) {
32                     SecurityContextHolder.getContext().setAuthentication(
33                             new UsernamePasswordAuthenticationToken(ud, null, ud.getAuthorities()));
34                 }
35             } catch (Exception ignored) {
```

```
                // invalid/expired token: request continues unauthenticated and is rejected by security rules
            }
        }
        chain.doFilter(req, res);
    }
}
```

backend/src/main/java/com/foodie/security/JwtService.java

```
package com.foodie.security;

import com.foodie.entity.User;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import javax.crypto.SecretKey;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

@Service
public class JwtService {
    private final SecretKey key;
    private final long expirationMs;

    public JwtService(@Value("${app.jwt.secret}") String secret, @Value("${app.jwt.expiration-ms}") long expirationMs) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expirationMs = expirationMs;
    }

    public String generate(User user) {
        Date now = new Date();
        return Jwts.builder().subject(user.getEmail()).claim("role", user.getRole().name())
                .issuedAt(now).expiration(new Date(now.getTime() + expirationMs)).signWith(key).compact();
    }

    public String extractEmail(String token) {
        return claims(token).getSubject();
    }

    private Claims claims(String token) {
        return Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload();
    }
}
```

backend/src/main/java/com/foodie/service/AuthService.java

```
package com.foodie.service;

import com.foodie.dto.Dtos.*;
import com.foodie.entity.User;
import com.foodie.exception.BadRequestException;
import com.foodie.repository.UserRepository;
import com.foodie.security.JwtService;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class AuthService {
    private final UserRepository users;
    private final PasswordEncoder encoder;
    private final AuthenticationManager authManager;
    private final JwtService jwt;

    public AuthService(UserRepository users, PasswordEncoder encoder, AuthenticationManager authManager, JwtService jwt) {
        this.users = users;
        this.encoder = encoder;
        this.authManager = authManager;
        this.jwt = jwt;
    }

    public AuthResponse register(RegisterRequest req) {
        String email = req.email().trim().toLowerCase();
        if (users.existsByEmail(email)) throw new BadRequestException("Email already registered");
        User u = new User();
        u.setName(req.name());
        u.setEmail(email);
        u.setPassword(encoder.encode(req.password()));
        u.setPhone(req.phone());
        u.setAddress(req.address());
```

```
        u.setRole(User.Role.USER);
        return toResponse(users.save(u));
    }

    public AuthResponse login(LoginRequest req) {
        String email = req.email().trim().toLowerCase();
        authManager.authenticate(new UsernamePasswordAuthenticationToken(email, req.password()));
        return toResponse(users.findByEmail(email).orElseThrow());
    }

    private AuthResponse toResponse(User u) {
        return new AuthResponse(jwt.generate(u), u.getId(), u.getName(), u.getEmail(), u.getRole().name());
    }
}
```

backend/src/main/java/com/foodie/service/DishService.java

```
package com.foodie.service;

import com.foodie.dto.Dtos.DishRequest;
import com.foodie.entity.Dish;
import com.foodie.exception.NotFoundException;
import com.foodie.repository.DishRepository;
import com.foodie.repository.RestaurantRepository;
import java.math.BigDecimal;
import java.util.List;
import org.springframework.stereotype.Service;

@Service
public class DishService {
    private final DishRepository dishes;
    private final RestaurantRepository restaurants;

    public DishService(DishRepository dishes, RestaurantRepository restaurants) {
        this.dishes = dishes;
        this.restaurants = restaurants;
    }

    public List<Dish> search(Long restaurantId, String category, Boolean veg, BigDecimal maxPrice, String q) {
        String cat = (category == null || category.isBlank()) ? null : category;
        String query = (q == null || q.isBlank()) ? null : q.trim();
        return dishes.search(restaurantId, cat, veg, maxPrice, query);
    }

    public Dish findById(Long id) {
        return dishes.findById(id).orElseThrow(() -> new NotFoundException("Dish " + id + " not found"));
    }

    public Dish create(DishRequest req) {
        return dishes.save(apply(new Dish(), req));
    }

    public Dish update(Long id, DishRequest req) {
        return dishes.save(apply(findById(id), req));
    }

    public void delete(Long id) {
        dishes.delete(findById(id));
    }

    private Dish apply(Dish d, DishRequest req) {
        d.setRestaurant(restaurants.findById(req.restaurantId())
                .orElseThrow(() -> new NotFoundException("Restaurant " + req.restaurantId() + " not found")));
        d.setName(req.name());
        d.setDescription(req.description());
        d.setPrice(req.price());
        d.setCategory(req.category());
        d.setVeg(req.veg());
        d.setImageUrl(req.imageUrl());
        if (req.available() != null) d.setAvailable(req.available());
        return d;
    }
}
```

backend/src/main/java/com/foodie/service/OrderService.java

```
package com.foodie.service;

import com.foodie.dto.Dtos.OrderDto;
import com.foodie.dto.Dtos.OrderItemRequest;
import com.foodie.dto.Dtos.OrderRequest;
import com.foodie.entity.Dish;
import com.foodie.entity.OrderEntity;
```

```java
import com.foodie.entity.OrderEntity.Status;
import com.foodie.entity.OrderItem;
import com.foodie.entity.User;
import com.foodie.exception.BadRequestException;
import com.foodie.exception.NotFoundException;
import com.foodie.repository.DishRepository;
import com.foodie.repository.OrderRepository;
import com.foodie.repository.UserRepository;
import java.math.BigDecimal;
import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {
    private final OrderRepository orders;
    private final DishRepository dishes;
    private final UserRepository users;

    public OrderService(OrderRepository orders, DishRepository dishes, UserRepository users) {
        this.orders = orders;
        this.dishes = dishes;
        this.users = users;
    }

    @Transactional
    public OrderDto placeOrder(String email, OrderRequest req) {
        User user = users.findByEmail(email).orElseThrow(() -> new NotFoundException("User not found"));
        OrderEntity order = new OrderEntity();
        order.setUser(user);
        order.setDeliveryAddress(req.deliveryAddress());
        BigDecimal total = BigDecimal.ZERO;
        for (OrderItemRequest ir : req.items()) {
            Dish dish = dishes.findById(ir.dishId())
                    .orElseThrow(() -> new NotFoundException("Dish " + ir.dishId() + " not found"));
            if (!dish.isAvailable()) throw new BadRequestException(dish.getName() + " is currently unavailable");
            OrderItem item = new OrderItem();
            item.setOrder(order);
            item.setDishId(dish.getId());
            item.setDishName(dish.getName());
            item.setRestaurantName(dish.getRestaurant().getName());
            item.setUnitPrice(dish.getPrice());
            item.setQuantity(ir.quantity());
            order.getItems().add(item);
            total = total.add(dish.getPrice().multiply(BigDecimal.valueOf(ir.quantity())));
        }
        order.setTotalAmount(total);
        return OrderDto.from(orders.save(order));
    }

    public List<OrderDto> myOrders(String email) {
        User user = users.findByEmail(email).orElseThrow(() -> new NotFoundException("User not found"));
        return orders.findByUserIdOrderByCreatedAtDesc(user.getId()).stream().map(OrderDto::from).toList();
    }

    public List<OrderDto> allOrders() {
        return orders.findAllByOrderByCreatedAtDesc().stream().map(OrderDto::from).toList();
    }

    @Transactional
    public OrderDto cancel(String email, Long orderId) {
        OrderEntity o = get(orderId);
        if (!o.getUser().getEmail().equalsIgnoreCase(email)) throw new NotFoundException("Order " + orderId + " not found");
        if (o.getStatus() != Status.PLACED && o.getStatus() != Status.CONFIRMED) {
            throw new BadRequestException("Order can no longer be cancelled (status: " + o.getStatus() + ")");
        }
        o.setStatus(Status.CANCELLED);
        return OrderDto.from(orders.save(o));
    }

    @Transactional
    public OrderDto updateStatus(Long orderId, Status status) {
        OrderEntity o = get(orderId);
        o.setStatus(status);
        return OrderDto.from(orders.save(o));
    }

    private OrderEntity get(Long id) {
        return orders.findById(id).orElseThrow(() -> new NotFoundException("Order " + id + " not found"));
    }
}
```

backend/src/main/java/com/foodie/service/RestaurantService.java

```java
package com.foodie.service;

```

```java
import com.foodie.entity.Restaurant;
import com.foodie.entity.Dish;
import com.foodie.exception.NotFoundException;
import com.foodie.repository.DishRepository;
import com.foodie.repository.RestaurantRepository;
import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class RestaurantService {
    private final RestaurantRepository restaurants;
    private final DishRepository dishes;

    public RestaurantService(RestaurantRepository restaurants, DishRepository dishes) {
        this.restaurants = restaurants;
        this.dishes = dishes;
    }

    public List<Restaurant> findAll() { return restaurants.findAll(); }

    public Restaurant findById(Long id) {
        return restaurants.findById(id).orElseThrow(() -> new NotFoundException("Restaurant " + id + " not found"));
    }

    public Restaurant create(Restaurant r) {
        r.setId(null);
        return restaurants.save(r);
    }

    public Restaurant update(Long id, Restaurant r) {
        Restaurant existing = findById(id);
        existing.setName(r.getName());
        existing.setCuisine(r.getCuisine());
        existing.setCity(r.getCity());
        existing.setAddress(r.getAddress());
        existing.setImageUrl(r.getImageUrl());
        existing.setRating(r.getRating());
        return restaurants.save(existing);
    }

    @Transactional
    public void delete(Long id) {
        Restaurant r = findById(id);
        List<Dish> owned = dishes.search(id, null, null, null, null);
        dishes.deleteAll(owned);
        restaurants.delete(r);
    }
}
```

backend/src/main/java/com/foodie/service/UserService.java

```java
package com.foodie.service;

import com.foodie.dto.Dtos.UserDto;
import com.foodie.dto.Dtos.UserUpdateRequest;
import com.foodie.entity.User;
import com.foodie.exception.BadRequestException;
import com.foodie.exception.NotFoundException;
import com.foodie.repository.UserRepository;
import java.util.List;
import org.springframework.stereotype.Service;

@Service
public class UserService {
    private final UserRepository users;

    public UserService(UserRepository users) { this.users = users; }

    public List<UserDto> findAll() { return users.findAll().stream().map(UserDto::from).toList(); }

    public UserDto findById(Long id) { return UserDto.from(get(id)); }

    public UserDto findByEmail(String email) {
        return UserDto.from(users.findByEmail(email).orElseThrow(() -> new NotFoundException("User not found")));
    }

    public UserDto update(Long id, UserUpdateRequest req) {
        User u = get(id);
        u.setName(req.name());
        u.setPhone(req.phone());
        u.setAddress(req.address());
        if (req.role() != null) u.setRole(req.role());
        if (req.active() != null) u.setActive(req.active());
        return UserDto.from(users.save(u));
    }
```

```
35
36      public UserDto updateProfile(String email, UserUpdateRequest req) {
37          User u = users.findByEmail(email).orElseThrow(() -> new NotFoundException("User not found"));
38          u.setName(req.name());
39          u.setPhone(req.phone());
40          u.setAddress(req.address());
41          return UserDto.from(users.save(u));
42      }
43
44      public void delete(Long id, String currentEmail) {
45          User u = get(id);
46          if (u.getEmail().equalsIgnoreCase(currentEmail)) throw new BadRequestException("You cannot delete your own account");
47          users.delete(u);
48      }
49
50      private User get(Long id) {
51          return users.findById(id).orElseThrow(() -> new NotFoundException("User " + id + " not found"));
52      }
53  }
```

backend/src/main/resources/application-h2.properties

```
1  spring.datasource.url=jdbc:h2:mem:foodie;DB_CLOSE_DELAY=-1
2  spring.datasource.username=sa
3  spring.datasource.password=
4  spring.jpa.hibernate.ddl-auto=update
5  spring.h2.console.enabled=true
```

backend/src/main/resources/application-mysql.properties

```
1  spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/foodie_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRe
   ieval=true&serverTimezone=UTC}
2  spring.datasource.username=${DB_USERNAME:root}
3  spring.datasource.password=${DB_PASSWORD:root}
4  spring.jpa.hibernate.ddl-auto=update
```

backend/src/main/resources/application.properties

```
1  server.port=8080
2  # Default profile = "h2" (in-memory, zero setup). Use: -Dspring.profiles.active=mysql for MySQL.
3  spring.profiles.default=h2
4
5  spring.jpa.open-in-view=false
6  spring.jackson.default-property-inclusion=non_null
7
8  app.jwt.secret=${JWT_SECRET:foodie-capstone-super-secret-key-change-me-in-production-0123456789}
9  app.jwt.expiration-ms=86400000
10
11 springdoc.swagger-ui.path=/swagger-ui.html
```

backend/src/test/java/com/foodie/service/DishServiceTest.java

```
1  package com.foodie.service;
2
3  import static org.junit.jupiter.api.Assertions.*;
4  import static org.mockito.ArgumentMatchers.any;
5  import static org.mockito.Mockito.*;
6
7  import com.foodie.dto.Dtos.DishRequest;
8  import com.foodie.entity.Dish;
9  import com.foodie.entity.Restaurant;
10 import com.foodie.exception.NotFoundException;
11 import com.foodie.repository.DishRepository;
12 import com.foodie.repository.RestaurantRepository;
13 import java.math.BigDecimal;
14 import java.util.List;
15 import java.util.Optional;
16 import org.junit.jupiter.api.Test;
17 import org.junit.jupiter.api.extension.ExtendWith;
18 import org.mockito.InjectMocks;
19 import org.mockito.Mock;
20 import org.mockito.junit.jupiter.MockitoExtension;
21
22 @ExtendWith(MockitoExtension.class)
23 class DishServiceTest {
24     @Mock DishRepository dishes;
25     @Mock RestaurantRepository restaurants;
```

```
    @InjectMocks DishService service;

    @Test
    void create_linksRestaurantAndSaves() {
        Restaurant r = new Restaurant();
        r.setId(1L);
        when(restaurants.findById(1L)).thenReturn(Optional.of(r));
        when(dishes.save(any(Dish.class))).thenAnswer(i -> i.getArgument(0));

        Dish d = service.create(new DishRequest(1L, "Pizza", "tasty", new BigDecimal("250"), "Pizza", true, null, null));

        assertEquals("Pizza", d.getName());
        assertSame(r, d.getRestaurant());
        assertTrue(d.isAvailable());
    }

    @Test
    void create_unknownRestaurant_throws() {
        when(restaurants.findById(9L)).thenReturn(Optional.empty());
        assertThrows(NotFoundException.class,
                () -> service.create(new DishRequest(9L, "X", null, BigDecimal.ONE, null, false, null, null)));
    }

    @Test
    void findById_missing_throws() {
        when(dishes.findById(3L)).thenReturn(Optional.empty());
        assertThrows(NotFoundException.class, () -> service.findById(3L));
    }

    @Test
    void search_blankFiltersBecomeNull() {
        when(dishes.search(null, null, true, null, null)).thenReturn(List.of(new Dish()));
        assertEquals(1, service.search(null, " ", true, null, "").size());
    }

    @Test
    void delete_removesDish() {
        Dish d = new Dish();
        when(dishes.findById(4L)).thenReturn(Optional.of(d));
        service.delete(4L);
        verify(dishes).delete(d);
    }
}
```

backend/src/test/java/com/foodie/service/OrderServiceTest.java

```
package com.foodie.service;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

import com.foodie.dto.Dtos.OrderDto;
import com.foodie.dto.Dtos.OrderItemRequest;
import com.foodie.dto.Dtos.OrderRequest;
import com.foodie.entity.*;
import com.foodie.exception.BadRequestException;
import com.foodie.exception.NotFoundException;
import com.foodie.repository.DishRepository;
import com.foodie.repository.OrderRepository;
import com.foodie.repository.UserRepository;
import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository orders;
    @Mock DishRepository dishes;
    @Mock UserRepository users;
    @InjectMocks OrderService service;

    User user;
    Dish dish;

    @BeforeEach
    void setUp() {
        user = new User();
        user.setId(1L);
        user.setName("Demo");
```

```
        user.setEmail("user@foodie.com");
        Restaurant r = new Restaurant();
        r.setName("Spice Garden");
        dish = new Dish();
        dish.setId(10L);
        dish.setName("Biryani");
        dish.setPrice(new BigDecimal("100"));
        dish.setRestaurant(r);
    }

    @Test
    void placeOrder_calculatesTotalAndStatusPlaced() {
        when(users.findByEmail("user@foodie.com")).thenReturn(Optional.of(user));
        when(dishes.findById(10L)).thenReturn(Optional.of(dish));
        when(orders.save(any(OrderEntity.class))).thenAnswer(i -> i.getArgument(0));

        OrderDto dto = service.placeOrder("user@foodie.com",
                new OrderRequest("Hyderabad", List.of(new OrderItemRequest(10L, 3))));

        assertEquals(0, new BigDecimal("300").compareTo(dto.totalAmount()));
        assertEquals("PLACED", dto.status());
        assertEquals(1, dto.items().size());
    }

    @Test
    void placeOrder_unavailableDish_throws() {
        dish.setAvailable(false);
        when(users.findByEmail("user@foodie.com")).thenReturn(Optional.of(user));
        when(dishes.findById(10L)).thenReturn(Optional.of(dish));

        assertThrows(BadRequestException.class, () -> service.placeOrder("user@foodie.com",
                new OrderRequest("addr", List.of(new OrderItemRequest(10L, 1)))));
        verify(orders, never()).save(any());
    }

    @Test
    void placeOrder_unknownDish_throwsNotFound() {
        when(users.findByEmail("user@foodie.com")).thenReturn(Optional.of(user));
        when(dishes.findById(99L)).thenReturn(Optional.empty());

        assertThrows(NotFoundException.class, () -> service.placeOrder("user@foodie.com",
                new OrderRequest("addr", List.of(new OrderItemRequest(99L, 1)))));
    }

    @Test
    void cancel_placedOrder_setsCancelled() {
        OrderEntity o = order(OrderEntity.Status.PLACED);
        when(orders.findById(5L)).thenReturn(Optional.of(o));
        when(orders.save(o)).thenReturn(o);

        assertEquals("CANCELLED", service.cancel("user@foodie.com", 5L).status());
    }

    @Test
    void cancel_deliveredOrder_throws() {
        when(orders.findById(5L)).thenReturn(Optional.of(order(OrderEntity.Status.DELIVERED)));
        assertThrows(BadRequestException.class, () -> service.cancel("user@foodie.com", 5L));
    }

    @Test
    void cancel_otherUsersOrder_throwsNotFound() {
        when(orders.findById(5L)).thenReturn(Optional.of(order(OrderEntity.Status.PLACED)));
        assertThrows(NotFoundException.class, () -> service.cancel("other@foodie.com", 5L));
    }

    @Test
    void updateStatus_changesStatus() {
        OrderEntity o = order(OrderEntity.Status.PLACED);
        when(orders.findById(5L)).thenReturn(Optional.of(o));
        when(orders.save(o)).thenReturn(o);

        assertEquals("DELIVERED", service.updateStatus(5L, OrderEntity.Status.DELIVERED).status());
    }

    private OrderEntity order(OrderEntity.Status status) {
        OrderEntity o = new OrderEntity();
        o.setId(5L);
        o.setUser(user);
        o.setStatus(status);
        return o;
    }
}
```

backend/src/test/java/com/foodie/service/UserAuthServiceTest.java

```
package com.foodie.service;
```

```
 2
 3  import static org.junit.jupiter.api.Assertions.*;
 4  import static org.mockito.ArgumentMatchers.any;
 5  import static org.mockito.Mockito.*;
 6
 7  import com.foodie.dto.Dtos.AuthResponse;
 8  import com.foodie.dto.Dtos.RegisterRequest;
 9  import com.foodie.dto.Dtos.UserUpdateRequest;
10  import com.foodie.entity.User;
11  import com.foodie.exception.BadRequestException;
12  import com.foodie.repository.UserRepository;
13  import com.foodie.security.JwtService;
14  import java.util.Optional;
15  import org.junit.jupiter.api.Test;
16  import org.junit.jupiter.api.extension.ExtendWith;
17  import org.mockito.Mock;
18  import org.mockito.junit.jupiter.MockitoExtension;
19  import org.springframework.security.authentication.AuthenticationManager;
20  import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
21
22  @ExtendWith(MockitoExtension.class)
23  class UserAuthServiceTest {
24      @Mock UserRepository users;
25      @Mock AuthenticationManager authManager;
26
27      @Test
28      void register_hashesPasswordAndIssuesToken() {
29          JwtService jwt = new JwtService("unit-test-secret-key-that-is-long-enough-1234567890", 60000);
30          AuthService auth = new AuthService(users, new BCryptPasswordEncoder(), authManager, jwt);
31          when(users.existsByEmail("a@b.com")).thenReturn(false);
32          when(users.save(any(User.class))).thenAnswer(i -> i.getArgument(0));
33
34          AuthResponse res = auth.register(new RegisterRequest("A", "A@B.com", "secret1", null, null));
35
36          assertEquals("USER", res.role());
37          assertEquals("a@b.com", jwt.extractEmail(res.token()));
38      }
39
40      @Test
41      void register_duplicateEmail_throws() {
42          AuthService auth = new AuthService(users, new BCryptPasswordEncoder(), authManager,
43                  new JwtService("unit-test-secret-key-that-is-long-enough-1234567890", 60000));
44          when(users.existsByEmail("a@b.com")).thenReturn(true);
45          assertThrows(BadRequestException.class, () -> auth.register(new RegisterRequest("A", "a@b.com", "secret1", null, null)))
46      }
47
48      @Test
49      void deleteOwnAccount_throws() {
50          UserService service = new UserService(users);
51          User u = new User();
52          u.setId(1L);
53          u.setEmail("admin@foodie.com");
54          when(users.findById(1L)).thenReturn(Optional.of(u));
55          assertThrows(BadRequestException.class, () -> service.delete(1L, "admin@foodie.com"));
56          verify(users, never()).delete(any());
57      }
58
59      @Test
60      void update_changesRoleAndActive() {
61          UserService service = new UserService(users);
62          User u = new User();
63          u.setId(2L);
64          u.setName("Old");
65          u.setRole(User.Role.USER);
66          when(users.findById(2L)).thenReturn(Optional.of(u));
67          when(users.save(u)).thenReturn(u);
68
69          var dto = service.update(2L, new UserUpdateRequest("New", "999", "addr", User.Role.ADMIN, false));
70
71          assertEquals("New", dto.name());
72          assertEquals("ADMIN", dto.role());
73          assertFalse(dto.active());
74      }
75  }
```

database/schema.sql

```
 1  -- Reference schema for the Foodie database (MySQL 8).
 2  -- NOTE: The Spring Boot app creates/updates these tables automatically (spring.jpa.hibernate.ddl-auto=update)
 3  -- and seeds an admin user + sample restaurants/dishes on first start. Run this file only if you prefer manual setup.
 4
 5  CREATE DATABASE IF NOT EXISTS foodie_db;
 6  USE foodie_db;
 7
 8  CREATE TABLE IF NOT EXISTS users (
 9    id        BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
  name      VARCHAR(255) NOT NULL,
  email     VARCHAR(255) NOT NULL UNIQUE,
  password  VARCHAR(255) NOT NULL,          -- BCrypt hash
  phone     VARCHAR(255),
  address   VARCHAR(255),
  role      ENUM('ADMIN','USER') NOT NULL DEFAULT 'USER',
  active    BIT NOT NULL DEFAULT 1
);

CREATE TABLE IF NOT EXISTS restaurants (
  id        BIGINT AUTO_INCREMENT PRIMARY KEY,
  name      VARCHAR(255) NOT NULL,
  cuisine   VARCHAR(255),
  city      VARCHAR(255),
  address   VARCHAR(255),
  image_url VARCHAR(255),
  rating    DOUBLE NOT NULL DEFAULT 0
);

CREATE TABLE IF NOT EXISTS dishes (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  name          VARCHAR(255) NOT NULL,
  description   VARCHAR(500),
  price         DECIMAL(38,2) NOT NULL,
  category      VARCHAR(255),
  veg           BIT NOT NULL DEFAULT 0,
  image_url     VARCHAR(255),
  available     BIT NOT NULL DEFAULT 1,
  restaurant_id BIGINT NOT NULL,
  CONSTRAINT fk_dish_restaurant FOREIGN KEY (restaurant_id) REFERENCES restaurants(id)
);

CREATE TABLE IF NOT EXISTS orders (
  id               BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id          BIGINT NOT NULL,
  status           ENUM('PLACED','CONFIRMED','PREPARING','OUT_FOR_DELIVERY','DELIVERED','CANCELLED') NOT NULL,
  total_amount     DECIMAL(38,2) NOT NULL,
  delivery_address VARCHAR(255),
  created_at       DATETIME(6),
  CONSTRAINT fk_order_user FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE IF NOT EXISTS order_items (
  id              BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_id        BIGINT NOT NULL,
  dish_id         BIGINT,
  dish_name       VARCHAR(255),
  restaurant_name VARCHAR(255),
  unit_price      DECIMAL(38,2),
  quantity        INT NOT NULL,
  CONSTRAINT fk_item_order FOREIGN KEY (order_id) REFERENCES orders(id)
);
```

frontend/.gitignore

```
# See https://docs.github.com/get-started/getting-started-with-git/ignoring-files for more about ignoring files.

# Compiled output
/dist
/tmp
/out-tsc
/bazel-out

# Node
/node_modules
npm-debug.log
yarn-error.log

# IDEs and editors
.idea/
.project
.classpath
.c9/
*.launch
.settings/
*.sublime-workspace

# Visual Studio Code
.vscode/*
!.vscode/settings.json
!.vscode/tasks.json
!.vscode/launch.json
!.vscode/extensions.json
.history/*

# Miscellaneous
```

```
/.angular/cache
.sass-cache/
/connect.lock
/coverage
/libpeerconnection.log
testem.log
/typings

# System files
.DS_Store
Thumbs.db
```

frontend/Dockerfile

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx ng build --configuration production

FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist/frontend/browser /usr/share/nginx/html
EXPOSE 80
```

frontend/angular.json

```
{
  "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
  "version": 1,
  "cli": {
    "packageManager": "npm"
  },
  "newProjectRoot": "projects",
  "projects": {
    "frontend": {
      "projectType": "application",
      "schematics": {
        "@schematics/angular:class": {
          "skipTests": true
        },
        "@schematics/angular:component": {
          "skipTests": true
        },
        "@schematics/angular:directive": {
          "skipTests": true
        },
        "@schematics/angular:guard": {
          "skipTests": true
        },
        "@schematics/angular:interceptor": {
          "skipTests": true
        },
        "@schematics/angular:pipe": {
          "skipTests": true
        },
        "@schematics/angular:resolver": {
          "skipTests": true
        },
        "@schematics/angular:service": {
          "skipTests": true
        }
      },
      "root": "",
      "sourceRoot": "src",
      "prefix": "app",
      "architect": {
        "build": {
          "builder": "@angular-devkit/build-angular:application",
          "options": {
            "outputPath": "dist/frontend",
            "index": "src/index.html",
            "browser": "src/main.ts",
            "polyfills": [
              "zone.js"
            ],
            "tsConfig": "tsconfig.app.json",
            "assets": [
              {
                "glob": "**/*",
                "input": "public"
              }
```

```
            ],
            "styles": [
              "src/styles.css"
            ],
            "scripts": []
          },
          "configurations": {
            "production": {
              "budgets": [
                {
                  "type": "initial",
                  "maximumWarning": "500kB",
                  "maximumError": "1MB"
                },
                {
                  "type": "anyComponentStyle",
                  "maximumWarning": "2kB",
                  "maximumError": "4kB"
                }
              ],
              "outputHashing": "all"
            },
            "development": {
              "optimization": false,
              "extractLicenses": false,
              "sourceMap": true
            }
          },
          "defaultConfiguration": "production"
        },
        "serve": {
          "builder": "@angular-devkit/build-angular:dev-server",
          "configurations": {
            "production": {
              "buildTarget": "frontend:build:production"
            },
            "development": {
              "buildTarget": "frontend:build:development"
            }
          },
          "defaultConfiguration": "development"
        },
        "extract-i18n": {
          "builder": "@angular-devkit/build-angular:extract-i18n"
        },
        "test": {
          "builder": "@angular-devkit/build-angular:karma",
          "options": {
            "polyfills": [
              "zone.js",
              "zone.js/testing"
            ],
            "tsConfig": "tsconfig.spec.json",
            "assets": [
              {
                "glob": "**/*",
                "input": "public"
              }
            ],
            "styles": [
              "src/styles.css"
            ],
            "scripts": []
          }
        }
      }
    }
  }
}
```

frontend/nginx.conf

```
server {
  listen 80;
  root /usr/share/nginx/html;
  index index.html;
  location / {
    try_files $uri $uri/ /index.html;
  }
}
```

frontend/package.json

```
{
  "name": "frontend",
  "version": "0.0.0",
  "scripts": {
    "ng": "ng",
    "start": "ng serve",
    "build": "ng build",
    "watch": "ng build --watch --configuration development",
    "test": "ng test"
  },
  "private": true,
  "dependencies": {
    "@angular/animations": "^18.2.0",
    "@angular/common": "^18.2.0",
    "@angular/compiler": "^18.2.0",
    "@angular/core": "^18.2.0",
    "@angular/forms": "^18.2.0",
    "@angular/platform-browser": "^18.2.0",
    "@angular/platform-browser-dynamic": "^18.2.0",
    "@angular/router": "^18.2.0",
    "rxjs": "~7.8.0",
    "tslib": "^2.3.0",
    "zone.js": "~0.14.10"
  },
  "devDependencies": {
    "@angular-devkit/build-angular": "^18.2.21",
    "@angular/cli": "^18.2.21",
    "@angular/compiler-cli": "^18.2.0",
    "@types/jasmine": "~5.1.0",
    "jasmine-core": "~5.2.0",
    "karma": "~6.4.0",
    "karma-chrome-launcher": "~3.2.0",
    "karma-coverage": "~2.2.0",
    "karma-jasmine": "~5.1.0",
    "karma-jasmine-html-reporter": "~2.1.0",
    "typescript": "~5.5.2"
  }
}
```

frontend/src/app/app.component.ts

```
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';
import { AuthService } from './core/services/auth.service';
import { CartService } from './core/services/cart.service';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav class="navbar">
      <a class="brand" routerLink="/">■ Foodie</a>
      <div class="links">
        @if (auth.isAdmin()) {
          <a routerLink="/admin/restaurants" routerLinkActive="active">Restaurants</a>
          <a routerLink="/admin/dishes" routerLinkActive="active">Dishes</a>
          <a routerLink="/admin/users" routerLinkActive="active">Users</a>
          <a routerLink="/admin/orders" routerLinkActive="active">Orders</a>
        } @else {
          <a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{exact: true}">Menu</a>
          @if (auth.isLoggedIn()) {
            <a routerLink="/orders" routerLinkActive="active">My Orders</a>
            <a routerLink="/cart" routerLinkActive="active">Cart ({{ cart.count() }})</a>
            <a routerLink="/profile" routerLinkActive="active">Profile</a>
          }
        }
      </div>
      <div class="auth">
        @if (auth.session(); as s) {
          <span class="muted">{{ s.name }} ({{ s.role }})</span>
          <button class="btn secondary" (click)="auth.logout()">Logout</button>
        } @else {
          <a routerLink="/login" class="btn">Login</a>
          <a routerLink="/register" class="btn secondary">Register</a>
        }
      </div>
    </nav>
    <main class="container"><router-outlet /></main>
  `
})
export class AppComponent {
  constructor(public auth: AuthService, public cart: CartService) {}
}
```

frontend/src/app/app.config.ts

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';
import { authInterceptor } from './core/guards/auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor]))
  ]
};
```

frontend/src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { adminGuard, authGuard } from './core/guards/auth.guard';

export const routes: Routes = [
  { path: '', loadComponent: () => import('./pages/user/home.component').then(m => m.HomeComponent) },
  { path: 'login', loadComponent: () => import('./pages/auth/login.component').then(m => m.LoginComponent) },
  { path: 'register', loadComponent: () => import('./pages/auth/register.component').then(m => m.RegisterComponent) },
  { path: 'cart', canActivate: [authGuard], loadComponent: () => import('./pages/user/cart.component').then(m => m.CartComponent
},
  { path: 'orders', canActivate: [authGuard], loadComponent: () => import('./pages/user/orders.component').then(m => m.OrdersCom
nent) },
  { path: 'profile', canActivate: [authGuard], loadComponent: () => import('./pages/user/profile.component').then(m => m.Profile
mponent) },
  {
    path: 'admin', canActivate: [adminGuard],
    children: [
      { path: '', pathMatch: 'full', redirectTo: 'restaurants' },
      { path: 'restaurants', loadComponent: () => import('./pages/admin/admin-restaurants.component').then(m => m.AdminRestauran
Component) },
      { path: 'dishes', loadComponent: () => import('./pages/admin/admin-dishes.component').then(m => m.AdminDishesComponent) },
      { path: 'users', loadComponent: () => import('./pages/admin/admin-users.component').then(m => m.AdminUsersComponent) },
      { path: 'orders', loadComponent: () => import('./pages/admin/admin-orders.component').then(m => m.AdminOrdersComponent) }
    ]
  },
  { path: '**', redirectTo: '' }
];
```

frontend/src/app/core/config.ts

```
export const API_URL = `${window.location.protocol}//${window.location.hostname}:8080/api`;
```

frontend/src/app/core/guards/auth.guard.ts

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  return auth.isLoggedIn() ? true : inject(Router).createUrlTree(['/login']);
};

export const adminGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  if (!auth.isLoggedIn()) return inject(Router).createUrlTree(['/login']);
  return auth.isAdmin() ? true : inject(Router).createUrlTree(['/']);
};
```

frontend/src/app/core/guards/auth.interceptor.ts

```
import { HttpErrorResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { AuthService } from '../services/auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.token;
```

```
  const authed = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;
  return next(authed).pipe(
    catchError((err: HttpErrorResponse) => {
      // An expired/invalid token on a protected call forces re-login
      if (err.status === 401 && token && !req.url.includes('/auth/')) auth.logout();
      return throwError(() => err);
    })
  );
};

export function errorMessage(err: any, fallback = 'Something went wrong'): string {
  return err?.error?.message ?? (err?.status === 0 ? 'Cannot reach the server' : fallback);
}
```

frontend/src/app/core/models.ts

```
export interface AuthResponse { token: string; id: number; name: string; email: string; role: 'ADMIN' | 'USER'; }

export interface User {
  id: number; name: string; email: string; phone?: string; address?: string;
  role: 'ADMIN' | 'USER'; active: boolean;
}

export interface Restaurant {
  id?: number; name: string; cuisine?: string; city?: string; address?: string; imageUrl?: string; rating: number;
}

export interface Dish {
  id?: number; name: string; description?: string; price: number; category?: string; veg: boolean;
  imageUrl?: string; available: boolean; restaurant?: Restaurant;
}

export interface DishRequest {
  restaurantId: number; name: string; description?: string; price: number; category?: string;
  veg: boolean; imageUrl?: string; available: boolean;
}

export interface CartItem { dish: Dish; quantity: number; }

export type OrderStatus = 'PLACED' | 'CONFIRMED' | 'PREPARING' | 'OUT_FOR_DELIVERY' | 'DELIVERED' | 'CANCELLED';
export const ORDER_STATUSES: OrderStatus[] =
  ['PLACED', 'CONFIRMED', 'PREPARING', 'OUT_FOR_DELIVERY', 'DELIVERED', 'CANCELLED'];

export interface OrderItem { dishId: number; dishName: string; restaurantName: string; unitPrice: number; quantity: number; }

export interface Order {
  id: number; userId: number; userName: string; status: OrderStatus; totalAmount: number;
  deliveryAddress: string; createdAt: string; items: OrderItem[];
}
```

frontend/src/app/core/services/api.services.ts

```
import { HttpClient, HttpParams } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { API_URL } from '../config';
import { Dish, DishRequest, Order, OrderStatus, Restaurant, User } from '../models';

@Injectable({ providedIn: 'root' })
export class RestaurantService {
  private url = `${API_URL}/restaurants`;
  constructor(private http: HttpClient) {}
  getAll(): Observable<Restaurant[]> { return this.http.get<Restaurant[]>(this.url); }
  get(id: number): Observable<Restaurant> { return this.http.get<Restaurant>(`${this.url}/${id}`); }
  create(r: Restaurant): Observable<Restaurant> { return this.http.post<Restaurant>(this.url, r); }
  update(id: number, r: Restaurant): Observable<Restaurant> { return this.http.put<Restaurant>(`${this.url}/${id}`, r); }
  delete(id: number): Observable<void> { return this.http.delete<void>(`${this.url}/${id}`); }
}

export interface DishFilter { restaurantId?: number | null; category?: string; veg?: boolean | null; maxPrice?: number | null; q
 string; }

@Injectable({ providedIn: 'root' })
export class DishService {
  private url = `${API_URL}/dishes`;
  constructor(private http: HttpClient) {}

  search(f: DishFilter = {}): Observable<Dish[]> {
    let params = new HttpParams();
    if (f.restaurantId) params = params.set('restaurantId', f.restaurantId);
    if (f.category) params = params.set('category', f.category);
    if (f.veg !== null && f.veg !== undefined) params = params.set('veg', f.veg);
    if (f.maxPrice) params = params.set('maxPrice', f.maxPrice);
```

```
    if (f.q) params = params.set('q', f.q);
    return this.http.get<Dish[]>(this.url, { params });
  }
  create(d: DishRequest): Observable<Dish> { return this.http.post<Dish>(this.url, d); }
  update(id: number, d: DishRequest): Observable<Dish> { return this.http.put<Dish>(`${this.url}/${id}`, d); }
  delete(id: number): Observable<void> { return this.http.delete<void>(`${this.url}/${id}`); }
}

@Injectable({ providedIn: 'root' })
export class OrderService {
  constructor(private http: HttpClient) {}
  place(deliveryAddress: string, items: { dishId: number; quantity: number }[]): Observable<Order> {
    return this.http.post<Order>(`${API_URL}/orders`, { deliveryAddress, items });
  }
  myOrders(): Observable<Order[]> { return this.http.get<Order[]>(`${API_URL}/orders/my`); }
  cancel(id: number): Observable<Order> { return this.http.put<Order>(`${API_URL}/orders/${id}/cancel`, {}); }
  allOrders(): Observable<Order[]> { return this.http.get<Order[]>(`${API_URL}/admin/orders`); }
  updateStatus(id: number, status: OrderStatus): Observable<Order> {
    return this.http.put<Order>(`${API_URL}/admin/orders/${id}/status`, { status });
  }
}

@Injectable({ providedIn: 'root' })
export class UserService {
  constructor(private http: HttpClient) {}
  getAll(): Observable<User[]> { return this.http.get<User[]>(`${API_URL}/admin/users`); }
  update(id: number, u: Partial<User>): Observable<User> { return this.http.put<User>(`${API_URL}/admin/users/${id}`, u); }
  delete(id: number): Observable<void> { return this.http.delete<void>(`${API_URL}/admin/users/${id}`); }
  me(): Observable<User> { return this.http.get<User>(`${API_URL}/users/me`); }
  updateMe(u: Partial<User>): Observable<User> { return this.http.put<User>(`${API_URL}/users/me`, u); }
}
```

frontend/src/app/core/services/auth.service.ts

```
import { HttpClient } from '@angular/common/http';
import { Injectable, computed, signal } from '@angular/core';
import { Router } from '@angular/router';
import { Observable, tap } from 'rxjs';
import { API_URL } from '../config';
import { AuthResponse } from '../models';

const KEY = 'foodie_auth';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly _session = signal<AuthResponse | null>(this.restore());
  readonly session = this._session.asReadonly();
  readonly isLoggedIn = computed(() => !!this._session());
  readonly isAdmin = computed(() => this._session()?.role === 'ADMIN');

  constructor(private http: HttpClient, private router: Router) {}

  login(email: string, password: string): Observable<AuthResponse> {
    return this.http.post<AuthResponse>(`${API_URL}/auth/login`, { email, password }).pipe(tap(r => this.save(r)));
  }

  register(body: { name: string; email: string; password: string; phone?: string; address?: string }): Observable<AuthResponse>
    return this.http.post<AuthResponse>(`${API_URL}/auth/register`, body).pipe(tap(r => this.save(r)));
  }

  logout(): void {
    localStorage.removeItem(KEY);
    this._session.set(null);
    this.router.navigate(['/login']);
  }

  get token(): string | null { return this._session()?.token ?? null; }

  private save(r: AuthResponse): void {
    localStorage.setItem(KEY, JSON.stringify(r));
    this._session.set(r);
  }

  private restore(): AuthResponse | null {
    try { return JSON.parse(localStorage.getItem(KEY) ?? 'null'); } catch { return null; }
  }
}
```

frontend/src/app/core/services/cart.service.ts

```
import { Injectable, computed, signal } from '@angular/core';
import { CartItem, Dish } from '../models';

```

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private readonly _items = signal<CartItem[]>(JSON.parse(localStorage.getItem('foodie_cart') ?? '[]'));
  readonly items = this._items.asReadonly();
  readonly count = computed(() => this._items().reduce((s, i) => s + i.quantity, 0));
  readonly total = computed(() => this._items().reduce((s, i) => s + i.quantity * i.dish.price, 0));

  add(dish: Dish): void {
    const items = [...this._items()];
    const found = items.find(i => i.dish.id === dish.id);
    if (found) found.quantity++; else items.push({ dish, quantity: 1 });
    this.set(items);
  }

  setQuantity(dishId: number | undefined, qty: number): void {
    this.set(this._items().map(i => i.dish.id === dishId ? { ...i, quantity: qty } : i).filter(i => i.quantity > 0));
  }

  remove(dishId: number | undefined): void { this.set(this._items().filter(i => i.dish.id !== dishId)); }

  clear(): void { this.set([]); }

  private set(items: CartItem[]): void {
    this._items.set(items);
    localStorage.setItem('foodie_cart', JSON.stringify(items));
  }
}
```

frontend/src/app/pages/admin/admin-dishes.component.ts

```
import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Dish, DishRequest, Restaurant } from '../../core/models';
import { errorMessage } from '../../core/guards/auth.interceptor';
import { DishService, RestaurantService } from '../../core/services/api.services';

@Component({
  selector: 'app-admin-dishes',
  standalone: true,
  imports: [FormsModule],
  template: `
    <h2>Product (dish) management</h2>
    <div class="card">
      <h3>{{ editingId ? 'Edit dish #' + editingId : 'Add dish' }}</h3>
      <form (ngSubmit)="save()" class="form-grid">
        <select name="restaurantId" [(ngModel)]="form.restaurantId" required>
          <option [ngValue]="0" disabled>Select restaurant</option>
          @for (r of restaurants; track r.id) { <option [ngValue]="r.id">{{ r.name }}</option> }
        </select>
        <input name="name" placeholder="Dish name" [(ngModel)]="form.name" required />
        <input name="price" type="number" min="0" placeholder="Price" [(ngModel)]="form.price" required />
        <input name="category" placeholder="Category" [(ngModel)]="form.category" />
        <input name="imageUrl" placeholder="Image URL" [(ngModel)]="form.imageUrl" />
        <input name="description" placeholder="Description" [(ngModel)]="form.description" />
        <label><input type="checkbox" name="veg" [(ngModel)]="form.veg" /> Veg</label>
        <label><input type="checkbox" name="available" [(ngModel)]="form.available" /> Available</label>
        <div>
          <button class="btn" type="submit">{{ editingId ? 'Update' : 'Add' }}</button>
          @if (editingId) { <button class="btn secondary" type="button" (click)="reset()">Cancel</button> }
        </div>
      </form>
      @if (error) { <p class="error">{{ error }}</p> }
    </div>
    <table>
      <tr><th>ID</th><th>Name</th><th>Restaurant</th><th>Category</th><th>Price</th><th>Veg</th><th>Available</th><th></th></tr>
      @for (d of dishes; track d.id) {
        <tr>
          <td>{{ d.id }}</td><td>{{ d.name }}</td><td>{{ d.restaurant?.name }}</td><td>{{ d.category }}</td>
          <td>■{{ d.price }}</td><td>{{ d.veg ? 'Yes' : 'No' }}</td><td>{{ d.available ? 'Yes' : 'No' }}</td>
          <td>
            <button class="btn secondary" (click)="edit(d)">Edit</button>
            <button class="btn danger" (click)="remove(d)">Delete</button>
          </td>
        </tr>
      }
    </table>
  `
})
export class AdminDishesComponent implements OnInit {
  dishes: Dish[] = [];
  restaurants: Restaurant[] = [];
  form: DishRequest = this.blank();
  editingId: number | null = null;
  error = '';

  constructor(private dishSvc: DishService, private restaurantSvc: RestaurantService) {}
```

```

  ngOnInit(): void {
    this.restaurantSvc.getAll().subscribe(r => this.restaurants = r);
    this.load();
  }

  load(): void { this.dishSvc.search().subscribe(d => this.dishes = d); }

  blank(): DishRequest { return { restaurantId: 0, name: '', price: 0, category: '', description: '', imageUrl: '', veg: true, a
ilable: true }; }

  edit(d: Dish): void {
    this.editingId = d.id!;
    this.form = { restaurantId: d.restaurant!.id!, name: d.name, price: d.price, category: d.category, description: d.descriptio
                  imageUrl: d.imageUrl, veg: d.veg, available: d.available };
  }

  reset(): void { this.editingId = null; this.form = this.blank(); }

  save(): void {
    this.error = '';
    if (!this.form.restaurantId) { this.error = 'Please select a restaurant'; return; }
    const req = this.editingId ? this.dishSvc.update(this.editingId, this.form) : this.dishSvc.create(this.form);
    req.subscribe({ next: () => { this.reset(); this.load(); }, error: e => this.error = errorMessage(e) });
  }

  remove(d: Dish): void {
    if (!confirm(`Delete ${d.name}?`)) return;
    this.dishSvc.delete(d.id!).subscribe({ next: () => this.load(), error: e => this.error = errorMessage(e) });
  }
}
```

frontend/src/app/pages/admin/admin-orders.component.ts

```
import { DatePipe } from '@angular/common';
import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { ORDER_STATUSES, Order, OrderStatus } from '../../core/models';
import { errorMessage } from '../../core/guards/auth.interceptor';
import { OrderService } from '../../core/services/api.services';

@Component({
  selector: 'app-admin-orders',
  standalone: true,
  imports: [FormsModule, DatePipe],
  template: `
    <h2>All orders</h2>
    @if (error) { <p class="error">{{ error }}</p> }
    <table>
      <tr><th>#</th><th>Customer</th><th>Items</th><th>Total</th><th>Placed</th><th>Status</th></tr>
      @for (o of orders; track o.id) {
        <tr>
          <td>{{ o.id }}</td><td>{{ o.userName }}</td>
          <td>@for (i of o.items; track $index) { <div>{{ i.quantity }} × {{ i.dishName }}</div> }</td>
          <td>■{{ o.totalAmount }}</td>
          <td>{{ o.createdAt | date: 'short' }}</td>
          <td>
            <select [ngModel]="o.status" (ngModelChange)="setStatus(o, $event)">
              @for (s of statuses; track s) { <option [value]="s">{{ s }}</option> }
            </select>
          </td>
        </tr>
      }
    </table>
  `
})
export class AdminOrdersComponent implements OnInit {
  orders: Order[] = [];
  statuses = ORDER_STATUSES;
  error = '';

  constructor(private svc: OrderService) {}

  ngOnInit(): void { this.load(); }

  load(): void { this.svc.allOrders().subscribe(o => this.orders = o); }

  setStatus(o: Order, status: OrderStatus): void {
    this.svc.updateStatus(o.id, status).subscribe({ next: () => this.load(), error: e => this.error = errorMessage(e) });
  }
}
```

frontend/src/app/pages/admin/admin-restaurants.component.ts

```ts
import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Restaurant } from '../../core/models';
import { errorMessage } from '../../core/guards/auth.interceptor';
import { RestaurantService } from '../../core/services/api.services';

@Component({
  selector: 'app-admin-restaurants',
  standalone: true,
  imports: [FormsModule],
  template: `
    <h2>Restaurant management</h2>
    <div class="card">
      <h3>{{ form.id ? 'Edit restaurant #' + form.id : 'Add restaurant' }}</h3>
      <form (ngSubmit)="save()" class="form-grid">
        <input name="name" placeholder="Name" [(ngModel)]="form.name" required />
        <input name="cuisine" placeholder="Cuisine" [(ngModel)]="form.cuisine" />
        <input name="city" placeholder="City" [(ngModel)]="form.city" />
        <input name="address" placeholder="Address" [(ngModel)]="form.address" />
        <input name="imageUrl" placeholder="Image URL" [(ngModel)]="form.imageUrl" />
        <input name="rating" type="number" min="0" max="5" step="0.1" placeholder="Rating" [(ngModel)]="form.rating" />
        <div>
          <button class="btn" type="submit">{{ form.id ? 'Update' : 'Add' }}</button>
          @if (form.id) { <button class="btn secondary" type="button" (click)="reset()">Cancel</button> }
        </div>
      </form>
      @if (error) { <p class="error">{{ error }}</p> }
    </div>
    <table>
      <tr><th>ID</th><th>Name</th><th>Cuisine</th><th>City</th><th>Rating</th><th></th></tr>
      @for (r of restaurants; track r.id) {
        <tr>
          <td>{{ r.id }}</td><td>{{ r.name }}</td><td>{{ r.cuisine }}</td><td>{{ r.city }}</td><td>{{ r.rating }}</td>
          <td>
            <button class="btn secondary" (click)="edit(r)">Edit</button>
            <button class="btn danger" (click)="remove(r)">Delete</button>
          </td>
        </tr>
      }
    </table>
  `
})
export class AdminRestaurantsComponent implements OnInit {
  restaurants: Restaurant[] = [];
  form: Restaurant = this.blank();
  error = '';

  constructor(private svc: RestaurantService) {}

  ngOnInit(): void { this.load(); }

  load(): void { this.svc.getAll().subscribe(r => this.restaurants = r); }

  blank(): Restaurant { return { name: '', cuisine: '', city: '', address: '', imageUrl: '', rating: 4 }; }

  edit(r: Restaurant): void { this.form = { ...r }; }

  reset(): void { this.form = this.blank(); }

  save(): void {
    this.error = '';
    const req = this.form.id ? this.svc.update(this.form.id, this.form) : this.svc.create(this.form);
    req.subscribe({ next: () => { this.reset(); this.load(); }, error: e => this.error = errorMessage(e) });
  }

  remove(r: Restaurant): void {
    if (!confirm(`Delete ${r.name} and all its dishes?`)) return;
    this.svc.delete(r.id!).subscribe({ next: () => this.load(), error: e => this.error = errorMessage(e) });
  }
}
```

frontend/src/app/pages/admin/admin-users.component.ts

```ts
import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { User } from '../../core/models';
import { errorMessage } from '../../core/guards/auth.interceptor';
import { AuthService } from '../../core/services/auth.service';
import { UserService } from '../../core/services/api.services';

@Component({
  selector: 'app-admin-users',
  standalone: true,
  imports: [FormsModule],
  template: `
    <h2>User management</h2>
```

```
    @if (error) { <p class="error">{{ error }}</p> }
    <table>
      <tr><th>ID</th><th>Name</th><th>Email</th><th>Phone</th><th>Role</th><th>Active</th><th></th></tr>
      @for (u of users; track u.id) {
        <tr>
          <td>{{ u.id }}</td><td>{{ u.name }}</td><td>{{ u.email }}</td><td>{{ u.phone }}</td>
          <td>
            <select [ngModel]="u.role" (ngModelChange)="changeRole(u, $event)" [disabled]="isSelf(u)">
              <option value="USER">USER</option><option value="ADMIN">ADMIN</option>
            </select>
          </td>
          <td>{{ u.active ? 'Yes' : 'Disabled' }}</td>
          <td>
            <button class="btn secondary" [disabled]="isSelf(u)" (click)="toggle(u)">{{ u.active ? 'Disable' : 'Enable' }}</butt
>
            <button class="btn danger" [disabled]="isSelf(u)" (click)="remove(u)">Delete</button>
          </td>
        </tr>
      }
    </table>
  `
})
export class AdminUsersComponent implements OnInit {
  users: User[] = [];
  error = '';

  constructor(private svc: UserService, private auth: AuthService) {}

  ngOnInit(): void { this.load(); }

  load(): void { this.svc.getAll().subscribe(u => this.users = u); }

  isSelf(u: User): boolean { return u.email === this.auth.session()?.email; }

  changeRole(u: User, role: 'ADMIN' | 'USER'): void { this.update(u, { ...u, role }); }

  toggle(u: User): void { this.update(u, { ...u, active: !u.active }); }

  private update(u: User, body: User): void {
    this.svc.update(u.id, body).subscribe({ next: () => { this.error = ''; this.load(); }, error: e => { this.error = errorMessa
(e); this.load(); } });
  }

  remove(u: User): void {
    if (!confirm(`Delete user ${u.email}?`)) return;
    this.svc.delete(u.id).subscribe({ next: () => this.load(), error: e => this.error = errorMessage(e) });
  }
}
```

frontend/src/app/pages/auth/login.component.ts

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { errorMessage } from '../../core/guards/auth.interceptor';
import { AuthService } from '../../core/services/auth.service';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [FormsModule, RouterLink],
  template: `
    <div class="card narrow">
      <h2>Login</h2>
      <form (ngSubmit)="submit()">
        <label>Email</label>
        <input name="email" type="email" [(ngModel)]="email" required />
        <label>Password</label>
        <input name="password" type="password" [(ngModel)]="password" required />
        @if (error) { <p class="error">{{ error }}</p> }
        <button class="btn" type="submit" [disabled]="loading">Login</button>
      </form>
      <p class="muted">New here? <a routerLink="/register">Create an account</a></p>
      <p class="muted small">Demo: admin&#64;foodie.com / admin123  |  user&#64;foodie.com / user123</p>
    </div>
  `
})
export class LoginComponent {
  email = '';
  password = '';
  error = '';
  loading = false;

  constructor(private auth: AuthService, private router: Router) {}

  submit(): void {
```

```
    this.loading = true;
    this.error = '';
    this.auth.login(this.email, this.password).subscribe({
      next: r => this.router.navigate([r.role === 'ADMIN' ? '/admin' : '/']),
      error: e => { this.loading = false; this.error = e.status === 401 ? 'Invalid email or password' : errorMessage(e); }
    });
  }
}
```

frontend/src/app/pages/auth/register.component.ts

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { errorMessage } from '../../core/guards/auth.interceptor';
import { AuthService } from '../../core/services/auth.service';

@Component({
  selector: 'app-register',
  standalone: true,
  imports: [FormsModule, RouterLink],
  template: `
    <div class="card narrow">
      <h2>Create account</h2>
      <form (ngSubmit)="submit()">
        <label>Name</label><input name="name" [(ngModel)]="f.name" required />
        <label>Email</label><input name="email" type="email" [(ngModel)]="f.email" required />
        <label>Password (min 6)</label><input name="password" type="password" [(ngModel)]="f.password" required minlength="6" />
        <label>Phone</label><input name="phone" [(ngModel)]="f.phone" />
        <label>Address</label><input name="address" [(ngModel)]="f.address" />
        @if (error) { <p class="error">{{ error }}</p> }
        <button class="btn" type="submit" [disabled]="loading">Register</button>
      </form>
      <p class="muted">Already registered? <a routerLink="/login">Login</a></p>
    </div>
  `
})
export class RegisterComponent {
  f = { name: '', email: '', password: '', phone: '', address: '' };
  error = '';
  loading = false;

  constructor(private auth: AuthService, private router: Router) {}

  submit(): void {
    this.loading = true;
    this.error = '';
    this.auth.register(this.f).subscribe({
      next: () => this.router.navigate(['/']),
      error: e => { this.loading = false; this.error = errorMessage(e); }
    });
  }
}
```

frontend/src/app/pages/user/cart.component.ts

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { errorMessage } from '../../core/guards/auth.interceptor';
import { CartService } from '../../core/services/cart.service';
import { OrderService, UserService } from '../../core/services/api.services';

@Component({
  selector: 'app-cart',
  standalone: true,
  imports: [FormsModule, RouterLink],
  template: `
    <h2>Your cart</h2>
    @if (cart.items().length === 0) {
      <p class="muted">Your cart is empty. <a routerLink="/">Browse the menu</a></p>
    } @else {
      <table>
        <tr><th>Dish</th><th>Price</th><th>Qty</th><th>Subtotal</th><th></th></tr>
        @for (i of cart.items(); track i.dish.id) {
          <tr>
            <td>{{ i.dish.name }}<br /><span class="muted small">{{ i.dish.restaurant?.name }}</span></td>
            <td>■{{ i.dish.price }}</td>
            <td><input class="qty" type="number" min="1" [ngModel]="i.quantity" (ngModelChange)="cart.setQuantity(i.dish.id, $ev
t)" /></td>
            <td>■{{ i.dish.price * i.quantity }}</td>
            <td><button class="btn danger" (click)="cart.remove(i.dish.id)">Remove</button></td>
          </tr>
```

```
        }
      </table>
      <h3>Total: ■{{ cart.total() }}</h3>
      <div class="card narrow">
        <label>Delivery address</label>
        <input [(ngModel)]="address" name="address" />
        @if (error) { <p class="error">{{ error }}</p> }
        <button class="btn" (click)="checkout()" [disabled]="loading">Place order</button>
      </div>
    }
  `
})
export class CartComponent {
  address = '';
  error = '';
  loading = false;

  constructor(public cart: CartService, private orders: OrderService, private users: UserService, private router: Router) {
    this.users.me().subscribe(u => this.address = u.address ?? '');
  }

  checkout(): void {
    if (!this.address.trim()) { this.error = 'Please enter a delivery address'; return; }
    this.loading = true;
    this.error = '';
    const items = this.cart.items().map(i => ({ dishId: i.dish.id!, quantity: i.quantity }));
    this.orders.place(this.address, items).subscribe({
      next: () => { this.cart.clear(); this.router.navigate(['/orders']); },
      error: e => { this.loading = false; this.error = errorMessage(e); }
    });
  }
}
```

frontend/src/app/pages/user/home.component.ts

```
import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { Dish, Restaurant } from '../../core/models';
import { AuthService } from '../../core/services/auth.service';
import { CartService } from '../../core/services/cart.service';
import { DishService, RestaurantService } from '../../core/services/api.services';

@Component({
  selector: 'app-home',
  standalone: true,
  imports: [FormsModule],
  template: `
    <h2>Restaurants</h2>
    <div class="chips">
      <span class="chip" [class.on]="!filter.restaurantId" (click)="selectRestaurant(null)">All</span>
      @for (r of restaurants; track r.id) {
        <span class="chip" [class.on]="filter.restaurantId === r.id" (click)="selectRestaurant(r.id!)">
          {{ r.name }} · {{ r.cuisine }} · ■{{ r.rating }}
        </span>
      }
    </div>

    <div class="filters card">
      <input placeholder="Search dishes..." [(ngModel)]="filter.q" (ngModelChange)="load()" />
      <select [(ngModel)]="vegChoice" (ngModelChange)="load()">
        <option value="">Veg & Non-veg</option>
        <option value="true">Veg only</option>
        <option value="false">Non-veg only</option>
      </select>
      <select [(ngModel)]="filter.category" (ngModelChange)="load()">
        <option value="">All categories</option>
        @for (c of categories; track c) { <option [value]="c">{{ c }}</option> }
      </select>
      <input type="number" min="0" placeholder="Max price" [(ngModel)]="filter.maxPrice" (ngModelChange)="load()" />
    </div>

    @if (message) { <p class="success">{{ message }}</p> }
    @if (dishes.length === 0) { <p class="muted">No dishes match your filters.</p> }
    <div class="grid">
      @for (d of dishes; track d.id) {
        <div class="card dish">
          @if (d.imageUrl) { <img [src]="d.imageUrl" [alt]="d.name" /> }
          <h3>{{ d.name }} <span [class]="d.veg ? 'veg' : 'nonveg'">●</span></h3>
          <p class="muted">{{ d.restaurant?.name }} · {{ d.category }}</p>
          <p>{{ d.description }}</p>
          <div class="row">
            <strong>■{{ d.price }}</strong>
            <button class="btn" [disabled]="!d.available" (click)="add(d)">{{ d.available ? 'Add to cart' : 'Unavailable' }}</bu
on>
          </div>
```

```
        </div>
      }
    </div>
  `
})
export class HomeComponent implements OnInit {
  restaurants: Restaurant[] = [];
  dishes: Dish[] = [];
  categories: string[] = [];
  filter: { restaurantId: number | null; category: string; maxPrice: number | null; q: string } =
    { restaurantId: null, category: '', maxPrice: null, q: '' };
  vegChoice = '';
  message = '';

  constructor(private restaurantSvc: RestaurantService, private dishSvc: DishService, private cart: CartService,
              private auth: AuthService, private router: Router) {}

  ngOnInit(): void {
    this.restaurantSvc.getAll().subscribe(r => this.restaurants = r);
    this.dishSvc.search().subscribe(all => this.categories = [...new Set(all.map(d => d.category).filter((c): c is string => !!c
]);
    this.load();
  }

  selectRestaurant(id: number | null): void { this.filter.restaurantId = id; this.load(); }

  load(): void {
    this.dishSvc.search({ ...this.filter, veg: this.vegChoice === '' ? null : this.vegChoice === 'true' })
      .subscribe(d => this.dishes = d);
  }

  add(d: Dish): void {
    if (!this.auth.isLoggedIn()) { this.router.navigate(['/login']); return; }
    this.cart.add(d);
    this.message = `${d.name} added to cart`;
    setTimeout(() => this.message = '', 2000);
  }
}
```

frontend/src/app/pages/user/orders.component.ts

```
import { DatePipe } from '@angular/common';
import { Component, OnInit } from '@angular/core';
import { Order } from '../../core/models';
import { errorMessage } from '../../core/guards/auth.interceptor';
import { OrderService } from '../../core/services/api.services';

@Component({
  selector: 'app-orders',
  standalone: true,
  imports: [DatePipe],
  template: `
    <h2>My order history</h2>
    @if (error) { <p class="error">{{ error }}</p> }
    @if (orders.length === 0) { <p class="muted">You have not placed any orders yet.</p> }
    @for (o of orders; track o.id) {
      <div class="card">
        <div class="row">
          <strong>Order #{{ o.id }}</strong>
          <span class="badge {{ o.status }}">{{ o.status.replaceAll('_', ' ') }}</span>
        </div>
        <p class="muted small">{{ o.createdAt | date: 'medium' }} · {{ o.deliveryAddress }}</p>
        <ul>
          @for (i of o.items; track $index) { <li>{{ i.quantity }} × {{ i.dishName }} ({{ i.restaurantName }}) — ■{{ i.unitPrice
 i.quantity }}</li> }
        </ul>
        <div class="row">
          <strong>Total ■{{ o.totalAmount }}</strong>
          @if (o.status === 'PLACED' || o.status === 'CONFIRMED') {
            <button class="btn danger" (click)="cancel(o)">Cancel order</button>
          }
        </div>
      </div>
    }
  `
})
export class OrdersComponent implements OnInit {
  orders: Order[] = [];
  error = '';

  constructor(private svc: OrderService) {}

  ngOnInit(): void { this.load(); }

  load(): void { this.svc.myOrders().subscribe(o => this.orders = o); }

```

```
  cancel(o: Order): void {
    this.svc.cancel(o.id).subscribe({ next: () => this.load(), error: e => this.error = errorMessage(e) });
  }
}
```

frontend/src/app/pages/user/profile.component.ts

```
import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { User } from '../../core/models';
import { errorMessage } from '../../core/guards/auth.interceptor';
import { UserService } from '../../core/services/api.services';

@Component({
  selector: 'app-profile',
  standalone: true,
  imports: [FormsModule],
  template: `
    <div class="card narrow">
      <h2>My profile</h2>
      @if (user) {
        <form (ngSubmit)="save()">
          <label>Email</label><input [value]="user.email" disabled />
          <label>Name</label><input name="name" [(ngModel)]="user.name" required />
          <label>Phone</label><input name="phone" [(ngModel)]="user.phone" />
          <label>Address</label><input name="address" [(ngModel)]="user.address" />
          @if (msg) { <p class="success">{{ msg }}</p> }
          @if (error) { <p class="error">{{ error }}</p> }
          <button class="btn" type="submit">Save</button>
        </form>
      }
    </div>
  `
})
export class ProfileComponent implements OnInit {
  user?: User;
  msg = '';
  error = '';

  constructor(private svc: UserService) {}

  ngOnInit(): void { this.svc.me().subscribe(u => this.user = u); }

  save(): void {
    this.msg = this.error = '';
    this.svc.updateMe(this.user!).subscribe({
      next: u => { this.user = u; this.msg = 'Profile updated'; },
      error: e => this.error = errorMessage(e)
    });
  }
}
```

frontend/src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

frontend/src/styles.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: 'Segoe UI', Roboto, Arial, sans-serif; background: #f6f7f9; color: #222; }
.navbar { display: flex; align-items: center; gap: 24px; background: #e23744; color: #fff; padding: 12px 24px; flex-wrap: wrap; }
.navbar a { color: #fff; text-decoration: none; }
.navbar .brand { font-size: 1.4rem; font-weight: 700; }
.navbar .links { display: flex; gap: 16px; flex: 1; }
.navbar .links a.active { border-bottom: 2px solid #fff; }
.navbar .auth { display: flex; align-items: center; gap: 8px; }
.navbar .muted { color: #ffe3e6; }
.container { max-width: 1100px; margin: 24px auto; padding: 0 16px; }
.card { background: #fff; border-radius: 10px; padding: 16px; margin-bottom: 16px; box-shadow: 0 1px 4px rgba(0,0,0,.08); }
.card.narrow { max-width: 440px; margin: 24px auto; }
.grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(250px, 1fr)); gap: 16px; }
.dish img { width: 100%; height: 150px; object-fit: cover; border-radius: 8px; }
.dish h3 { margin: 8px 0 0; }
.row { display: flex; justify-content: space-between; align-items: center; gap: 8px; }
.filters { display: flex; gap: 8px; flex-wrap: wrap; }
```

```
.filters input, .filters select { flex: 1; min-width: 140px; }
.form-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 8px; align-items: center; }
input, select { padding: 8px 10px; border: 1px solid #ccd; border-radius: 6px; font-size: 14px; width: 100%; }
input[type=checkbox] { width: auto; }
input.qty { width: 70px; }
label { display: block; margin: 8px 0 4px; font-size: 13px; color: #555; }
.btn { display: inline-block; background: #e23744; color: #fff; border: 0; padding: 8px 14px; border-radius: 6px; cursor: pointe
 text-decoration: none; font-size: 14px; margin-right: 4px; }
.btn:disabled { opacity: .5; cursor: not-allowed; }
.btn.secondary { background: #fff; color: #e23744; border: 1px solid #e23744; }
.btn.danger { background: #b42318; }
table { width: 100%; border-collapse: collapse; background: #fff; border-radius: 10px; overflow: hidden; box-shadow: 0 1px 4px r
a(0,0,0,.08); }
th, td { text-align: left; padding: 10px 12px; border-bottom: 1px solid #eee; vertical-align: top; }
th { background: #fafafa; }
.chips { display: flex; gap: 8px; flex-wrap: wrap; margin-bottom: 12px; }
.chip { background: #fff; border: 1px solid #ddd; border-radius: 20px; padding: 6px 14px; cursor: pointer; font-size: 13px; }
.chip.on { background: #e23744; color: #fff; border-color: #e23744; }
.muted { color: #777; } .small { font-size: 12px; }
.error { color: #b42318; } .success { color: #067647; }
.veg { color: #1a9c3c; } .nonveg { color: #b42318; }
.badge { padding: 3px 10px; border-radius: 12px; font-size: 12px; background: #eee; }
.badge.PLACED { background: #e0efff; } .badge.CONFIRMED { background: #e6e0ff; } .badge.PREPARING { background: #fff2cc; }
.badge.OUT_FOR_DELIVERY { background: #ffe0cc; } .badge.DELIVERED { background: #d9f7e3; } .badge.CANCELLED { background: #fde0e
 }
```

.gitignore

```
target/
node_modules/
dist/
.angular/
.idea/
*.iml
.vscode/
.DS_Store
*.log
```

Jenkinsfile

```
// Jenkins declarative pipeline: build -> test -> docker images -> push -> deploy to AWS EC2.
// Required Jenkins credentials:
//   dockerhub-creds  (Username/Password)  - Docker Hub login
//   ec2-ssh-key      (SSH Username with private key) - key for the EC2 instance
// Required environment: set EC2_HOST (e.g. ec2-user@<public-ip>) and DOCKER_USER below / in Jenkins.
pipeline {
  agent any
  environment {
    DOCKER_USER = 'your-dockerhub-username'
    EC2_HOST    = 'ec2-user@your-ec2-public-ip'
    TAG         = "${env.BUILD_NUMBER}"
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Backend: test & package') {
      steps {
        dir('backend') { sh 'mvn -B clean verify' }
      }
      post { always { junit 'backend/target/surefire-reports/*.xml' } }
    }
    stage('Frontend: build') {
      steps {
        dir('frontend') {
          sh 'npm ci'
          sh 'npx ng build --configuration production'
        }
      }
    }
    stage('Docker build') {
      steps {
        sh 'docker build -t $DOCKER_USER/foodie-backend:$TAG -t $DOCKER_USER/foodie-backend:latest backend'
        sh 'docker build -t $DOCKER_USER/foodie-frontend:$TAG -t $DOCKER_USER/foodie-frontend:latest frontend'
      }
    }
    stage('Docker push') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'dockerhub-creds', usernameVariable: 'U', passwordVariable: 'P')]) {
          sh 'echo $P | docker login -u $U --password-stdin'
          sh 'docker push $DOCKER_USER/foodie-backend:$TAG && docker push $DOCKER_USER/foodie-backend:latest'
          sh 'docker push $DOCKER_USER/foodie-frontend:$TAG && docker push $DOCKER_USER/foodie-frontend:latest'
        }
```

```
      }
    }
    stage('Deploy to EC2') {
      steps {
        sshagent(['ec2-ssh-key']) {
          sh 'scp -o StrictHostKeyChecking=no docker-compose.prod.yml $EC2_HOST:~/docker-compose.yml'
          sh '''ssh -o StrictHostKeyChecking=no $EC2_HOST "export DOCKER_USER=$DOCKER_USER && docker compose pull && docker compose up -d"'''
        }
      }
    }
  }
}
```

docker-compose.prod.yml

```
# Used on the AWS EC2 instance by the Jenkins pipeline (images are pulled from Docker Hub).
# Required env var: DOCKER_USER. Optional: JWT_SECRET, MYSQL_ROOT_PASSWORD.
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root}
      MYSQL_DATABASE: foodie_db
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-p${MYSQL_ROOT_PASSWORD:-root}"]
      interval: 10s
      timeout: 5s
      retries: 10
    restart: always

  backend:
    image: ${DOCKER_USER}/foodie-backend:latest
    environment:
      SPRING_PROFILES_ACTIVE: mysql
      DB_URL: jdbc:mysql://mysql:3306/foodie_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      DB_USERNAME: root
      DB_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root}
      JWT_SECRET: ${JWT_SECRET:-foodie-capstone-super-secret-key-change-me-in-production-0123456789}
    ports:
      - "8080:8080"
    depends_on:
      mysql:
        condition: service_healthy
    restart: always

  frontend:
    image: ${DOCKER_USER}/foodie-frontend:latest
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: always

volumes:
  mysql-data:
```

docker-compose.yml

```
services:
  mysql:
    image: mysql:8.0
    container_name: foodie-mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: foodie_db
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-proot"]
      interval: 10s
      timeout: 5s
      retries: 10

  backend:
    build: ./backend
    container_name: foodie-backend
    environment:
      SPRING_PROFILES_ACTIVE: mysql
```

```
      DB_URL: jdbc:mysql://mysql:3306/foodie_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      DB_USERNAME: root
      DB_PASSWORD: root
      JWT_SECRET: ${JWT_SECRET:-foodie-capstone-super-secret-key-change-me-in-production-0123456789}
    ports:
      - "8080:8080"
    depends_on:
      mysql:
        condition: service_healthy

  frontend:
    build: ./frontend
    container_name: foodie-frontend
    ports:
      - "80:80"
    depends_on:
      - backend

volumes:
  mysql-data:
```